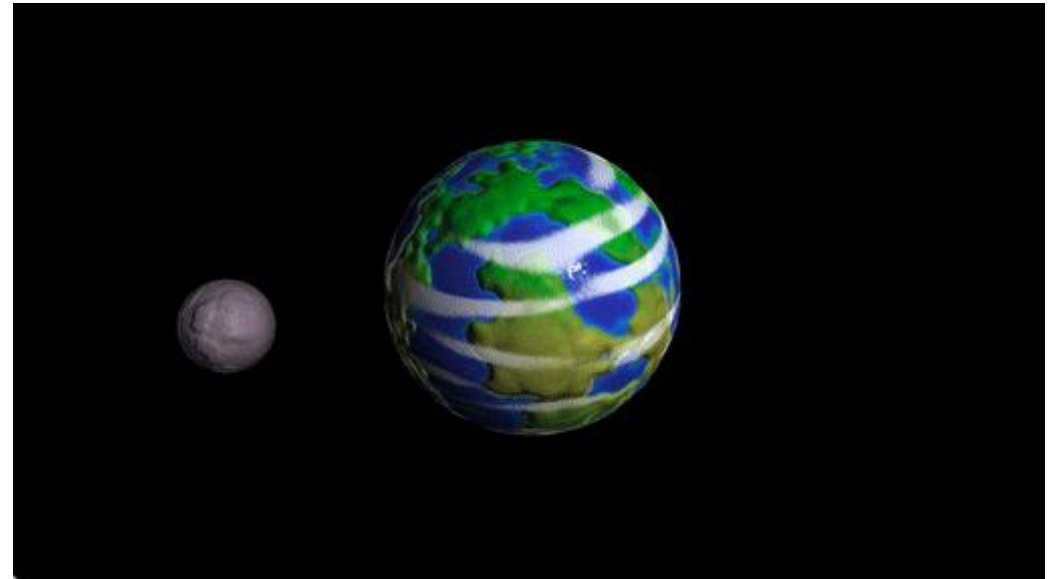


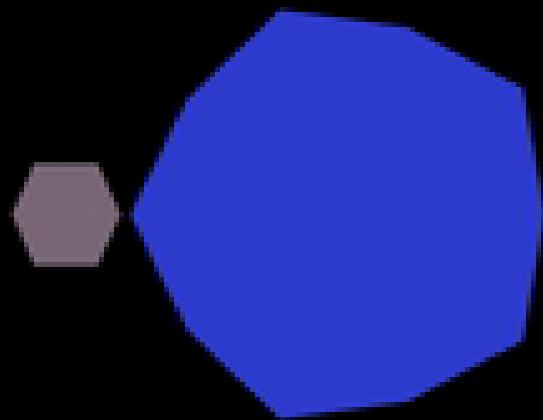
Transformations, Shaders & Rasterization

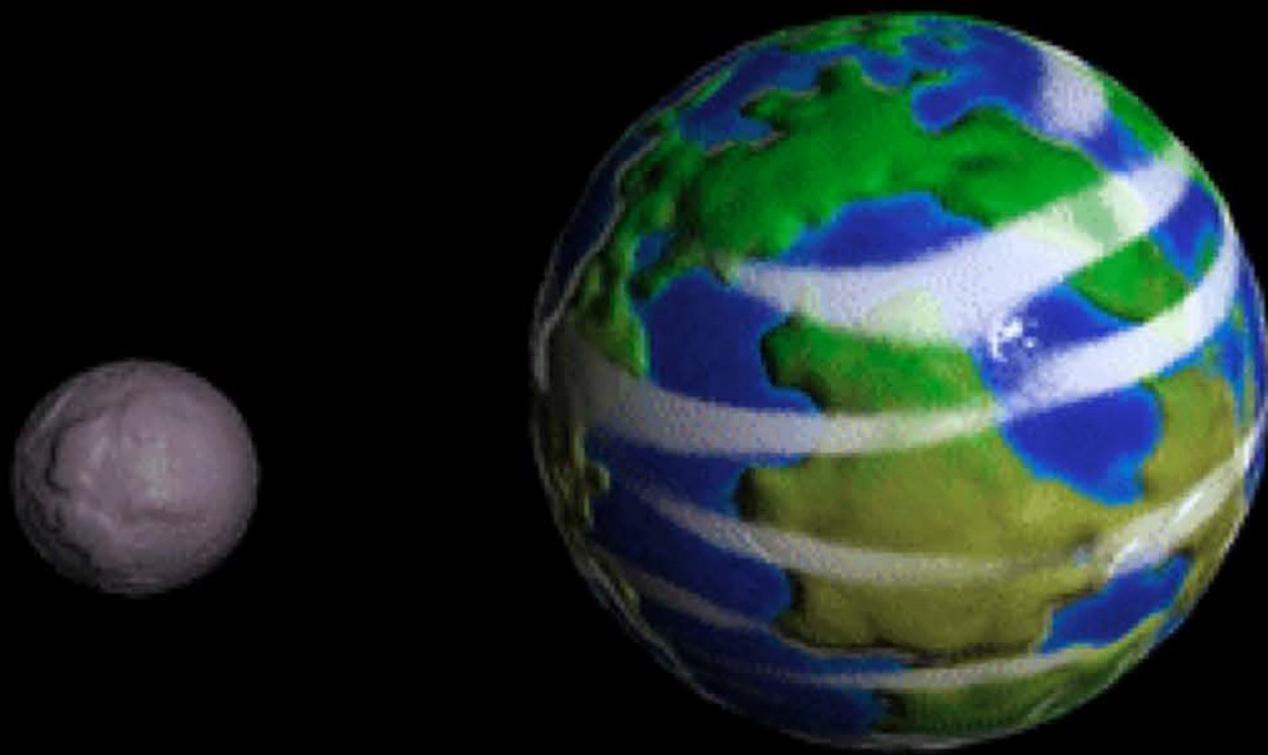


Agenda

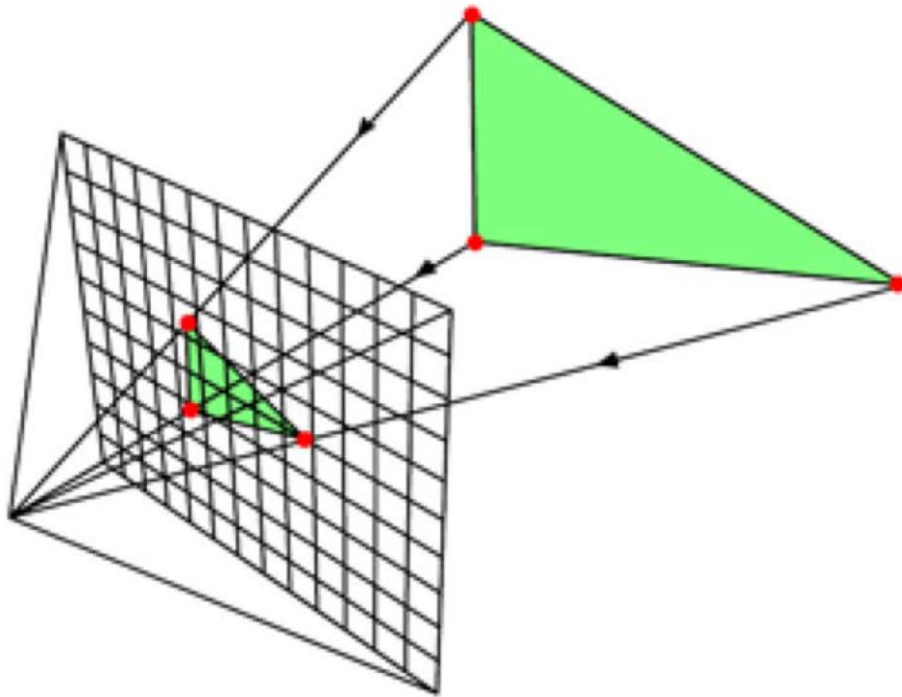
- Rasterization and the Modern Graphics Pipeline
- Transformations
- Shaders
- Normal and Bump Mapping
- Perlin Noise





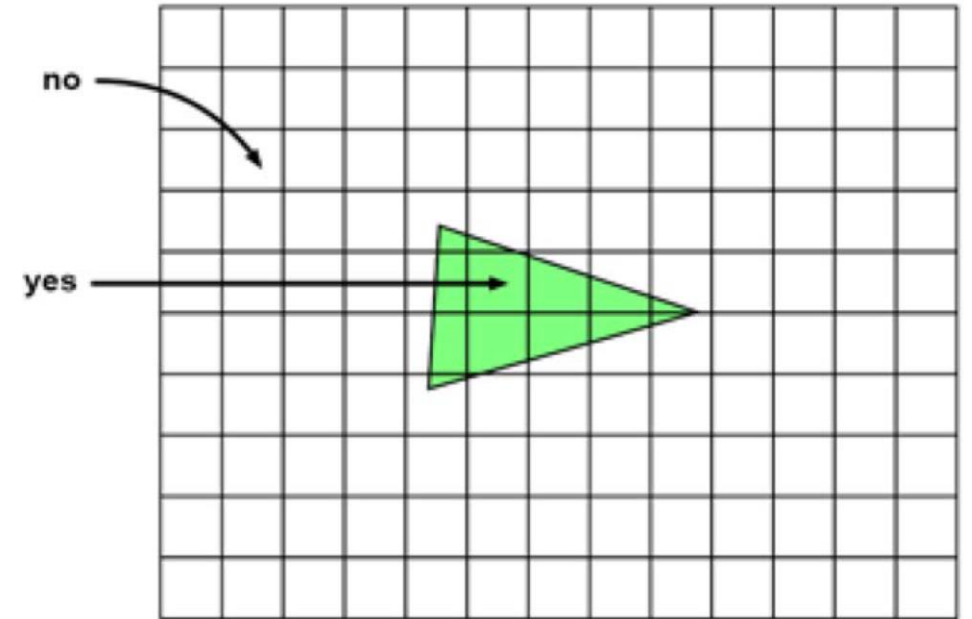


Rasterization



© www.scratchapixel.com

1. Project Vertices to Image Plane



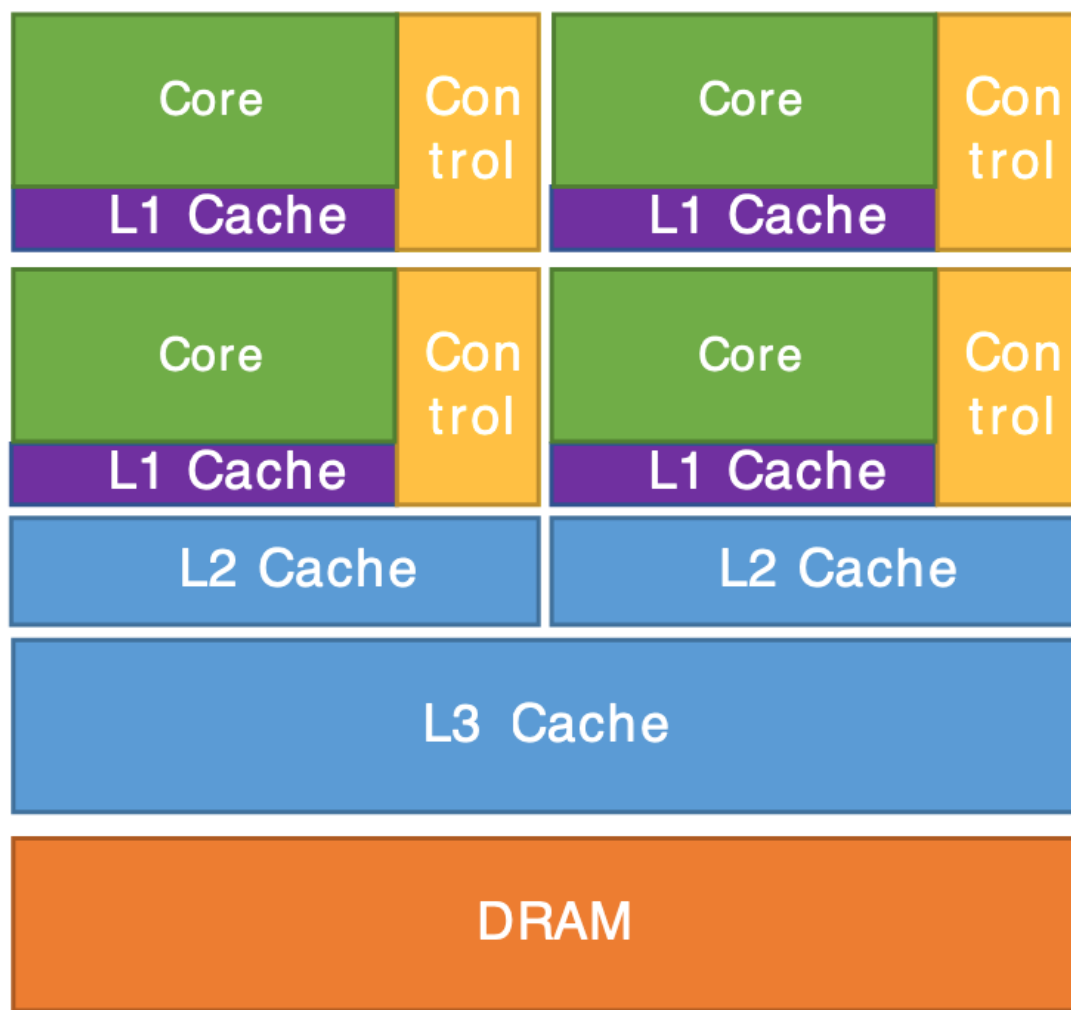
2. Turn on pixels inside triangle



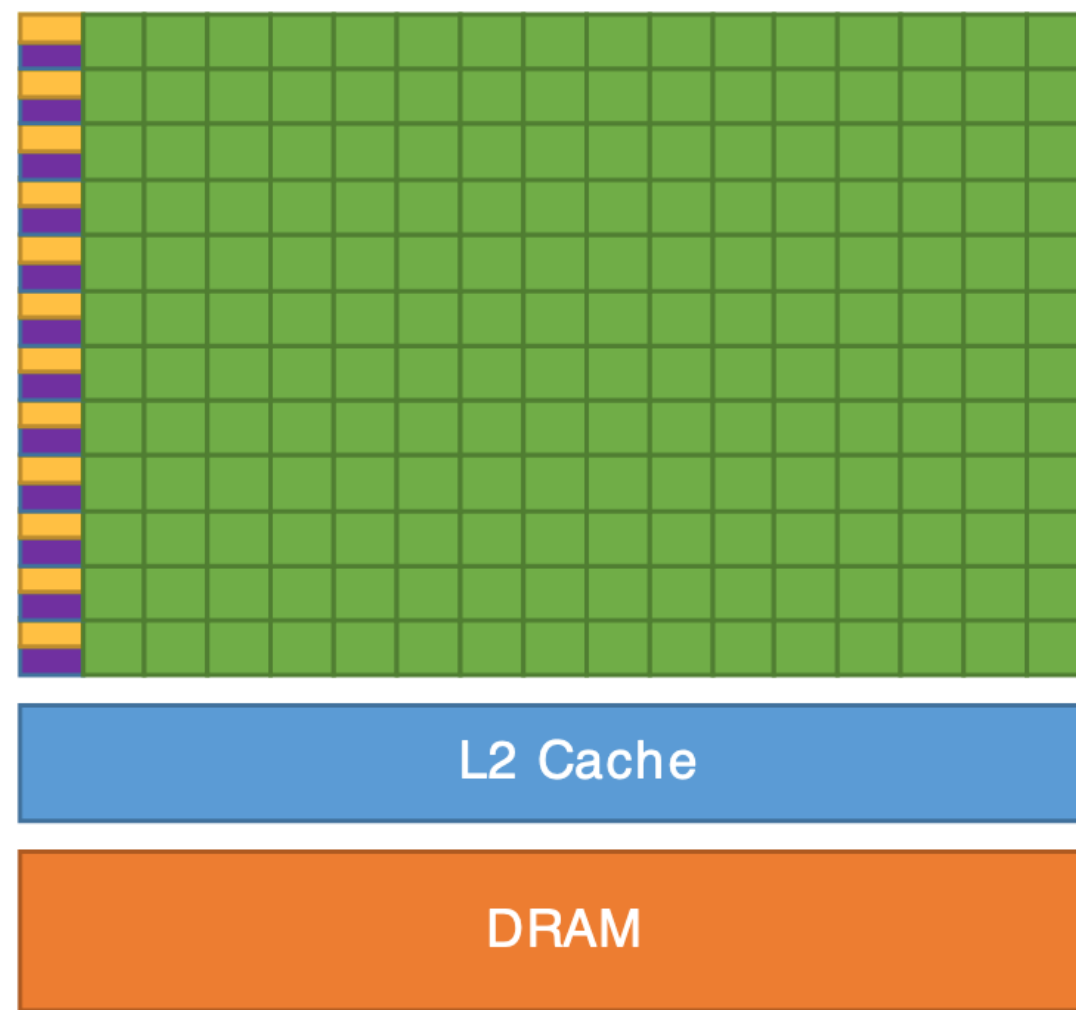
Rasterization

```
001 // rasterization algorithm
002 for (each triangle in scene) {
003     // STEP 1: project vertices of the triangle using perspective projection
004     Vec2f v0 = perspectiveProject(triangle[i].v0);
005     Vec2f v1 = perspectiveProject(triangle[i].v1);
006     Vec2f v2 = perspectiveProject(triangle[i].v2);
007     for (each pixel in image) {
008         // STEP 2: is this pixel contained in the projected image of the triangle
009         if (pixelContainedIn2DTriangle(v0, v1, v2, x, y)) {
010             image(x,y) = triangle[i].color;
011         }
012     }
013 }
```





CPU



GPU

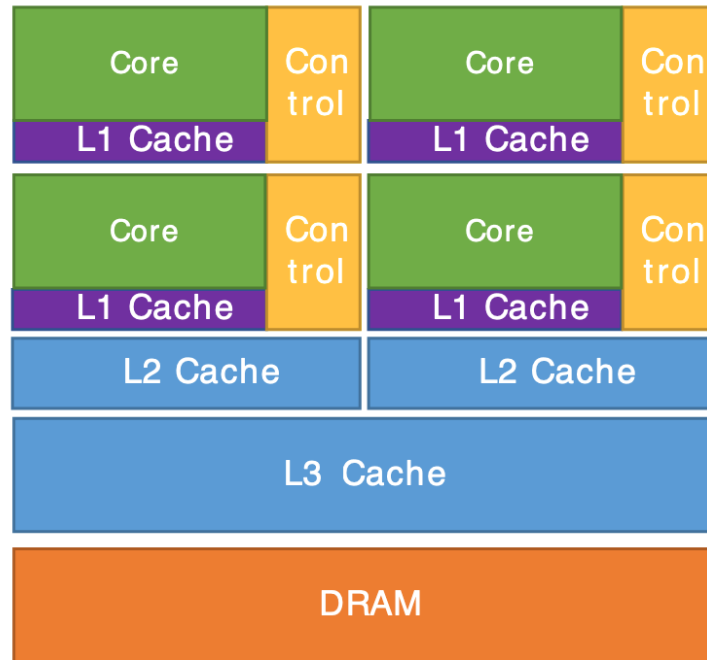
CPU

General Purpose, sequential tasks

Few cores

Optimized for **Latency**

Flexible control flow



CPU

GPU

Highly parallel, repetitive tasks

Many many cores

Optimized for **throughput**

Data-parallel workload



GPU

Thinking like GPU

CPU Program

```
struct Pixel {  
    float r, g, b;  
};  
  
void grayscaleCPU(vector<Pixel>& image) {  
    for (Pixel& p : image) {  
        float gray = convertGray(p);  
        p = {gray, gray, gray};  
    }  
}
```

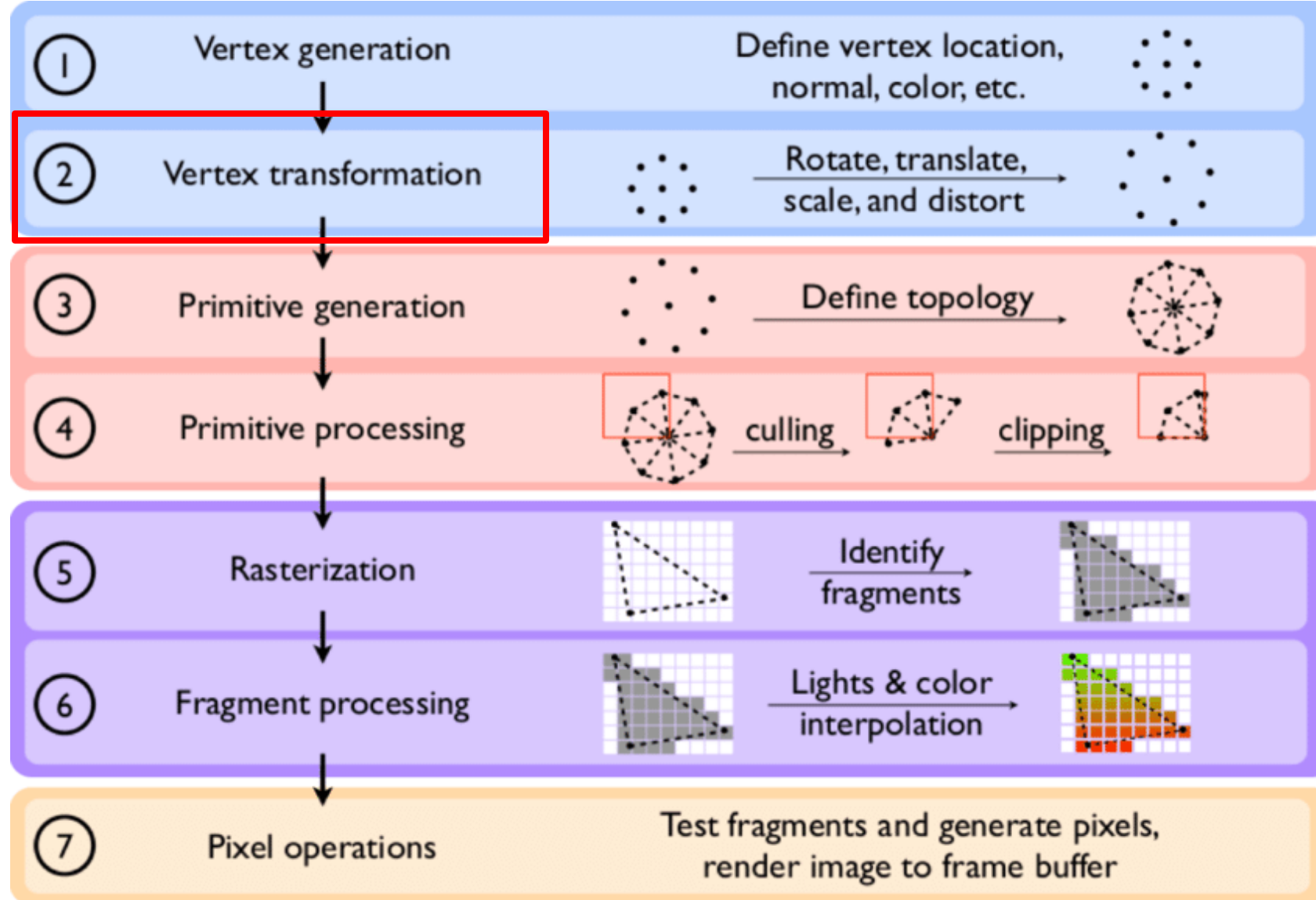
GPU Vertex Shader

```
#version 330 core  
layout(location = 0) in vec2 aPos;  
layout(location = 1) in vec2 aTexCoord;  
out vec2 TexCoord;  
  
void main() {  
    TexCoord = aTexCoord;  
    gl_Position = vec4(aPos, 0.0, 1.0);  
}
```

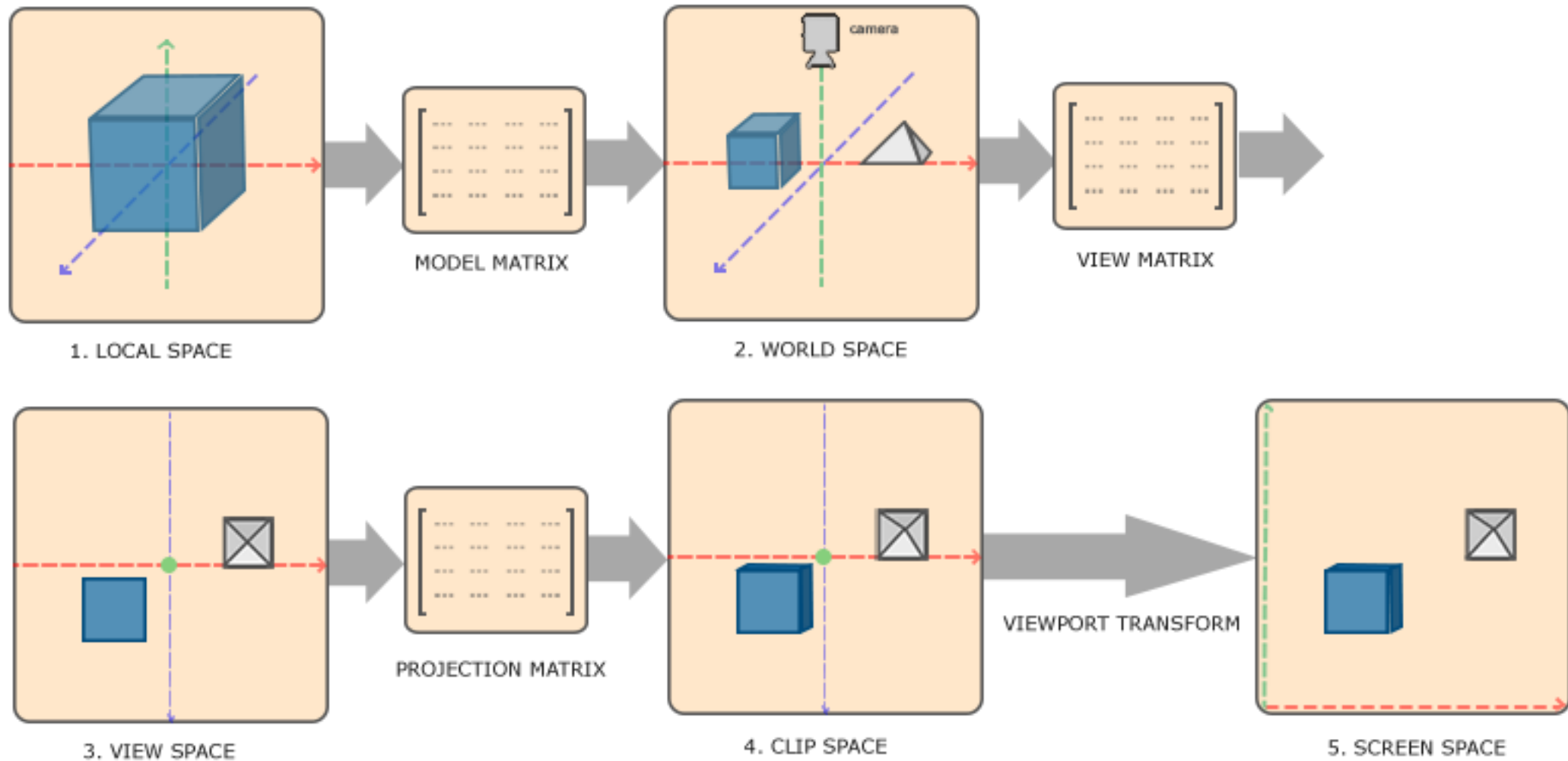
Program is applied for each vertex or fragment(pixel) provided.

Fragment Shader omitted

Modern Graphics Pipeline



Object space to Screen space



Transformations

Transformation/Deformation in Graphics:

A function f , mapping points/vectors to points/vectors.
simple transformations are usually invertible.

$$[x \ y]^T \begin{matrix} \xrightarrow{f} \\ \xleftarrow{f^{-1}} \end{matrix} [x' \ y']^T$$

Applications:

- Placing objects in a scene.
- Composing an object from parts.
- Animating objects.



Linear Transformations

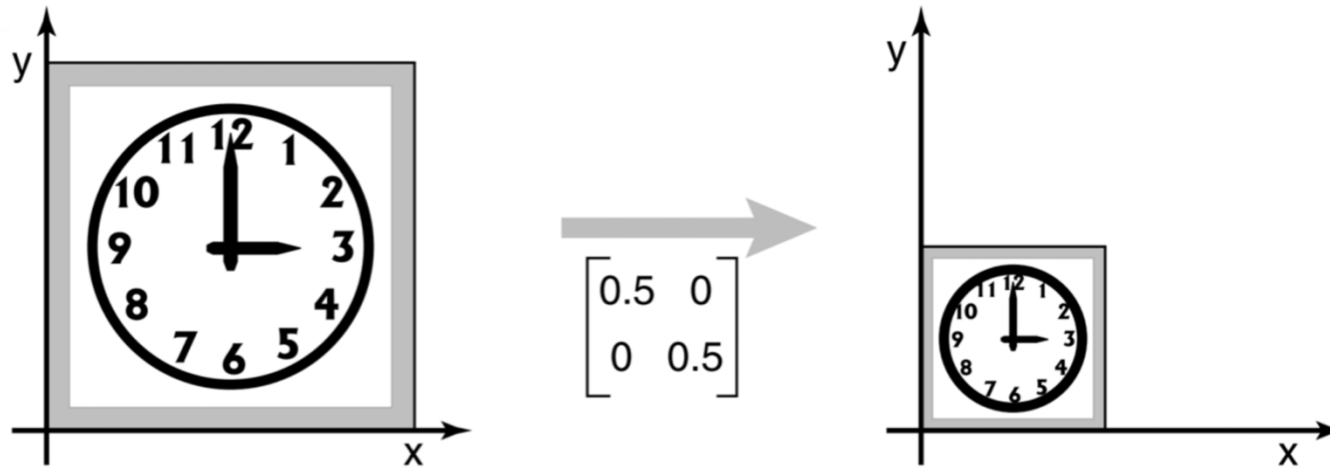
A transformation matrix A maps a point/vector $p=[x \ y]^T$ to Ap

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_{11}x + a_{12}y \\ a_{21}x + a_{22}y \end{bmatrix}$$



2D Linear Transformations - Scale

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} s_x x \\ s_y y \end{bmatrix}$$



When $s_x = s_y$ we say the scaling is uniform



2D Linear Transformations - Rotation

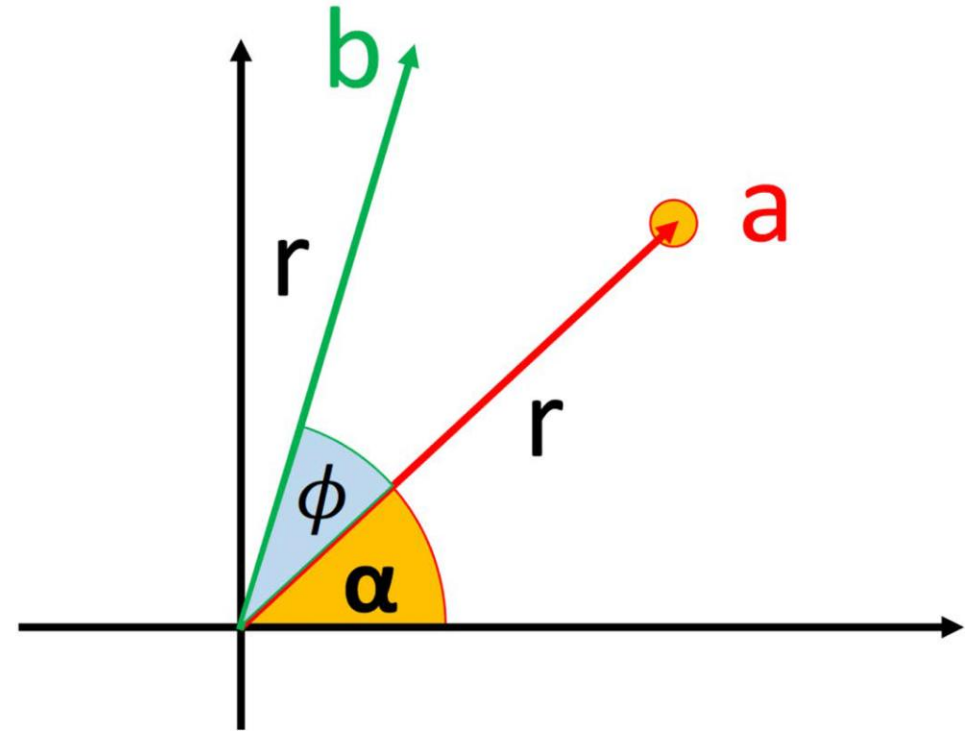
$$\text{rotate}(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$$

$$x = r \cos \alpha$$

$$y = r \sin \alpha$$

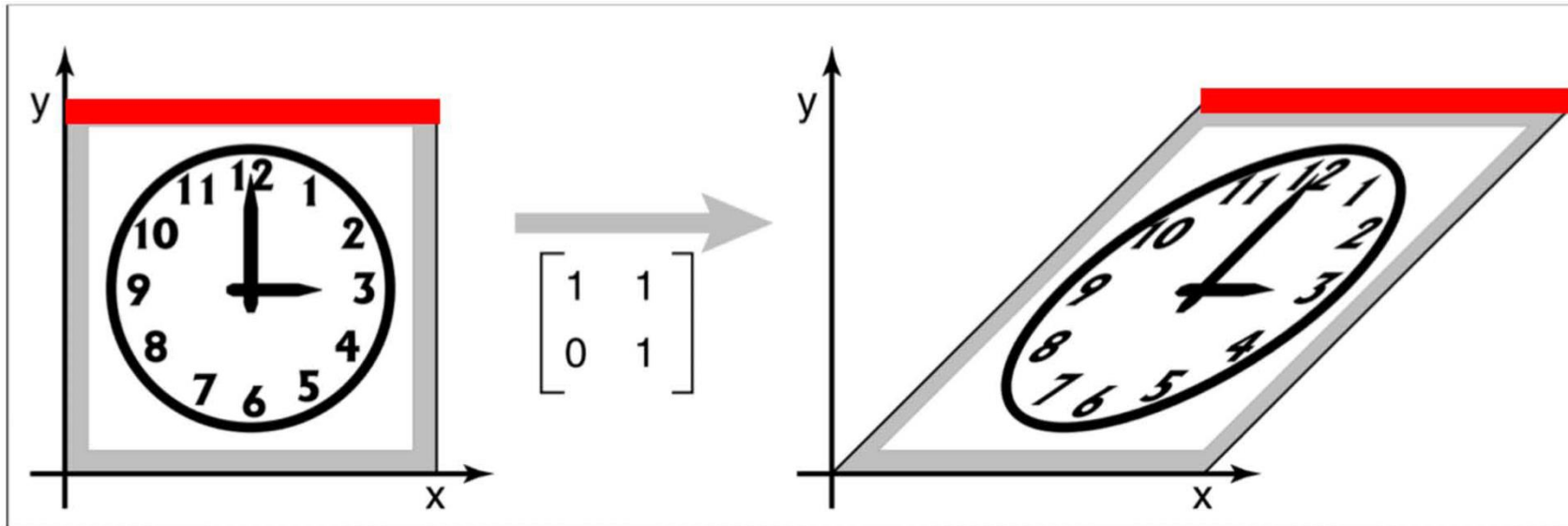
$$x' = r \cos(\alpha + \phi) = r (\cos \alpha \cos \phi - \sin \alpha \sin \phi) = x \cos \phi - y \sin \phi$$

$$y' = r \sin(\alpha + \phi) = r (\cos \alpha \sin \phi + \sin \alpha \cos \phi) = x \sin \phi + y \cos \phi$$



2D Linear Transformations - Shear

$$\begin{bmatrix} 1 & s \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + sy \\ y \end{bmatrix}$$



These are always the same length



2D Linear Transformations- Translation?

$$T \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \end{bmatrix}$$



2D Affine Transformations - Translation

$$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11}x + a_{12}y + t_x \\ a_{21}x + a_{22}y + t_y \\ 1 \end{bmatrix}$$

$$Ax + t = b$$



Cartesian $\Leftrightarrow$ Homogeneous Coordinates

Cartesian $[x \ y]^T \Rightarrow$ Homogeneous $[x \ y \ 1]^T$

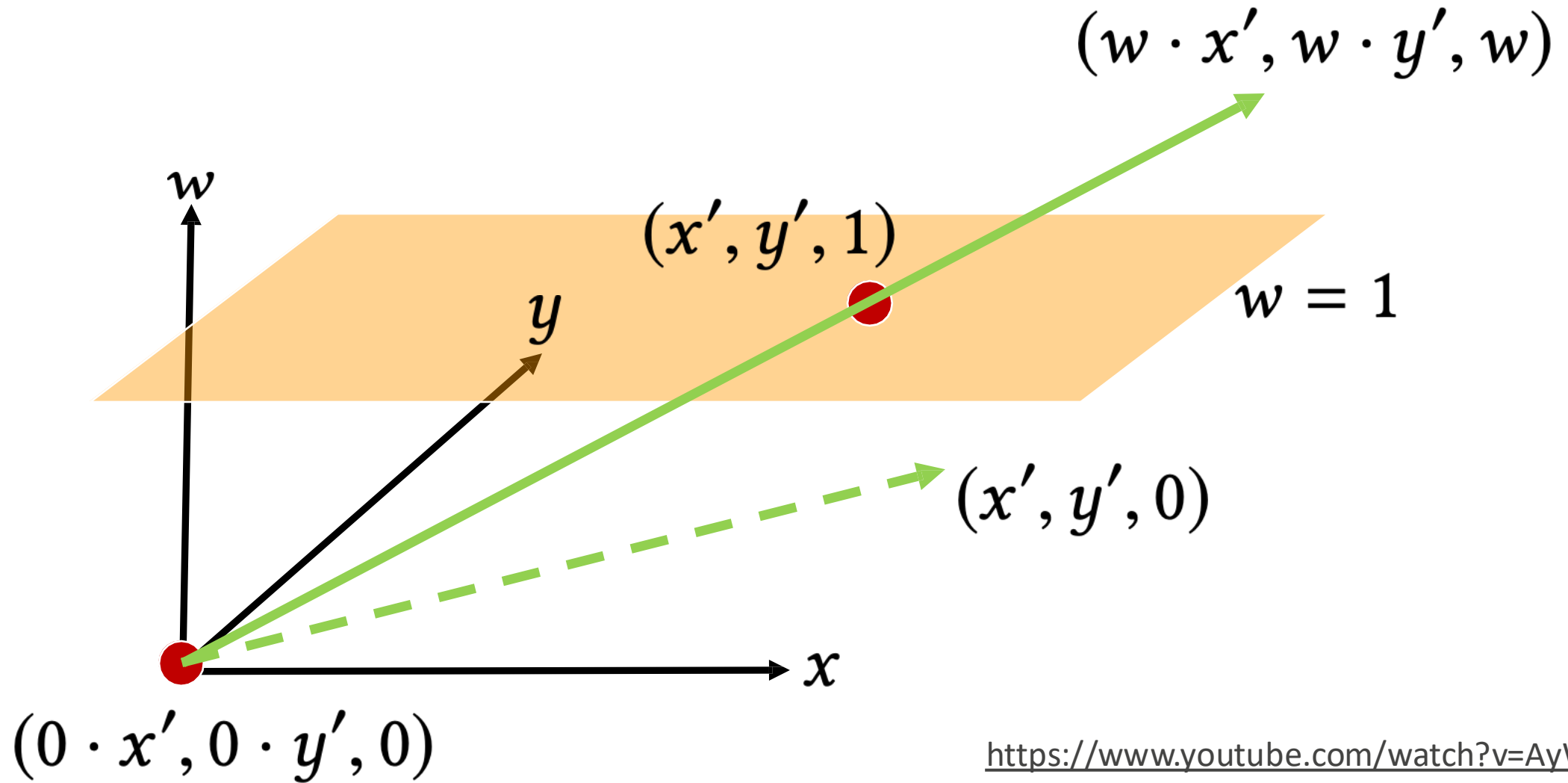
Homogeneous $[x \ y \ w]^T \Rightarrow$ Cartesian $[x/w \ y/w \ 1]^T$

Homogeneous points are equal if they represent the same Cartesian point. For eg. $[4 \ -6 \ 2]^T = [-6 \ 9 \ -3]^T$.

What about $w=0$?



Geometric Intuition



<https://www.youtube.com/watch?v=AyW4Y8APjK4>

<https://www.youtube.com/watch?v=2Snoepcmi9U>

<https://www.youtube.com/watch?v=Q2ultHa7GFQ>

Points at ∞ in Homogeneous Coordinates

$[x \ y \ w]^T$ with $w=0$ represent points at infinity, though with direction $[x \ y]^T$ and thus provide a natural representation for **vectors**, distinct from **points** in Homogeneous coordinates.

$$\begin{bmatrix} x \\ y \end{bmatrix}$$

Considered as a **vector** in 3D
homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ 0 \end{bmatrix}$$



Representing 2D transforms as a 3x3 matrix

Translate a point $[x \ y]^T$ by $[t_x \ t_y]^T$:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Rotate a point $[x \ y]^T$ by an angle t :

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos t & -\sin t & 0 \\ \sin t & \cos t & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Scale a point $[x \ y]^T$ by a factor $[s_x \ s_y]^T$

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$



Properties of 2D transforms

...these 3x3 transforms when invertible are called **Homographies**.

they map **lines** to **lines**.

i.e. points **p** on a line **l** transformed by a Homography **H** produce points **Hp** that also lie on a line.

...a more restricted set of transformations also preserve parallelism in lines. These are called **Affine** transforms.

...transforms that further preserve the angle between lines are called **Conformal**.

...transforms that additionally preserve the lengths of line segments are called **Rigid**.



Properties of 2D transforms

Homography (preserve lines)

Affine (preserve parallelism)
shear, scale

Conformal (preserve angles)
uniform scale

Rigid (preserve lengths)
rotate, translate



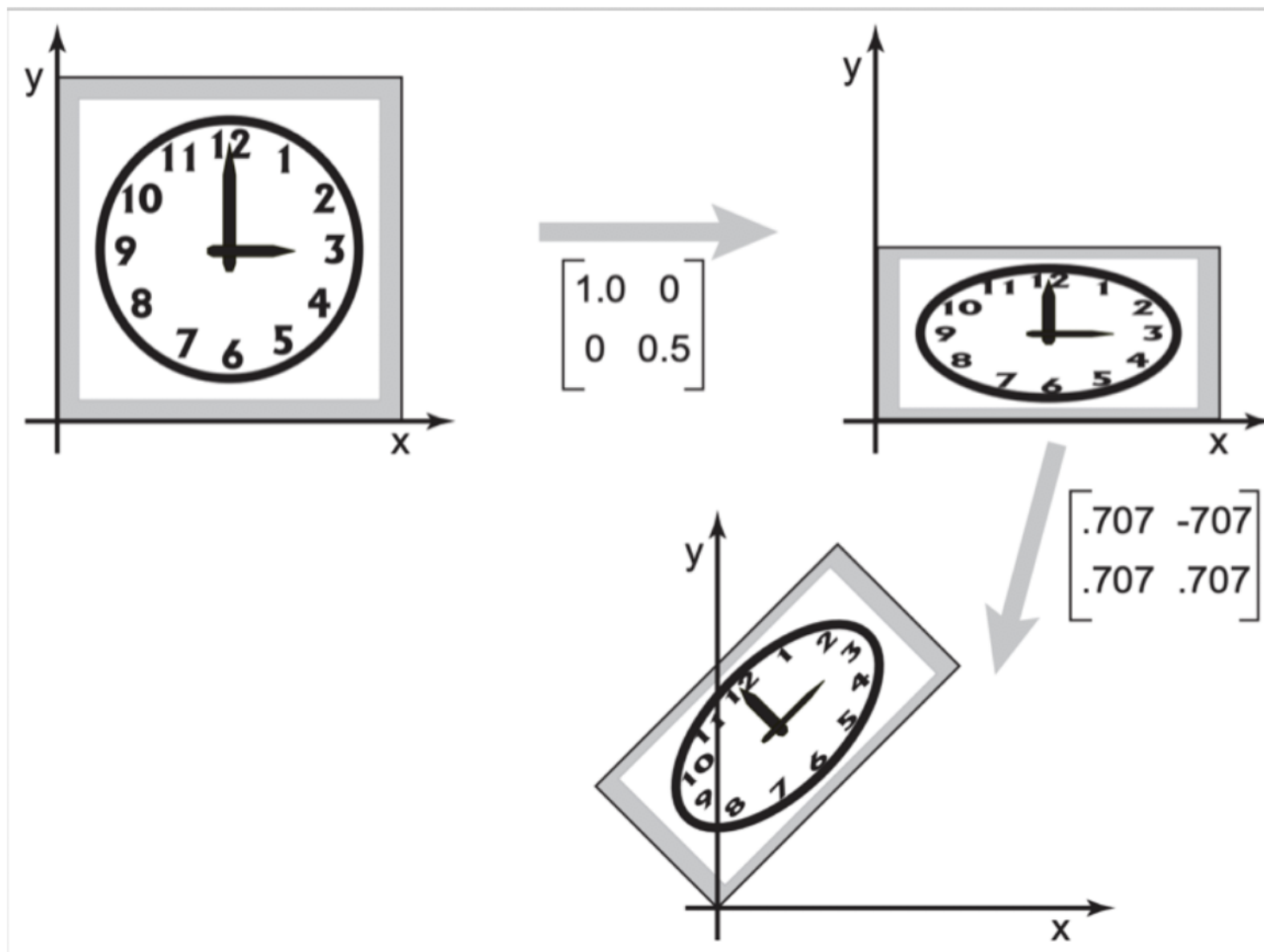
Affine properties: composition

Affine transforms are closed under composition.
i.e. Applying transform (A_1, t_1) (A_2, t_2) in sequence
results in an overall Affine transform.

$$p' = A_2 (A_1 p + t_1) + t_2 \Rightarrow (A_2 A_1) p + (A_2 t_1 + t_2)$$



Composing transformations

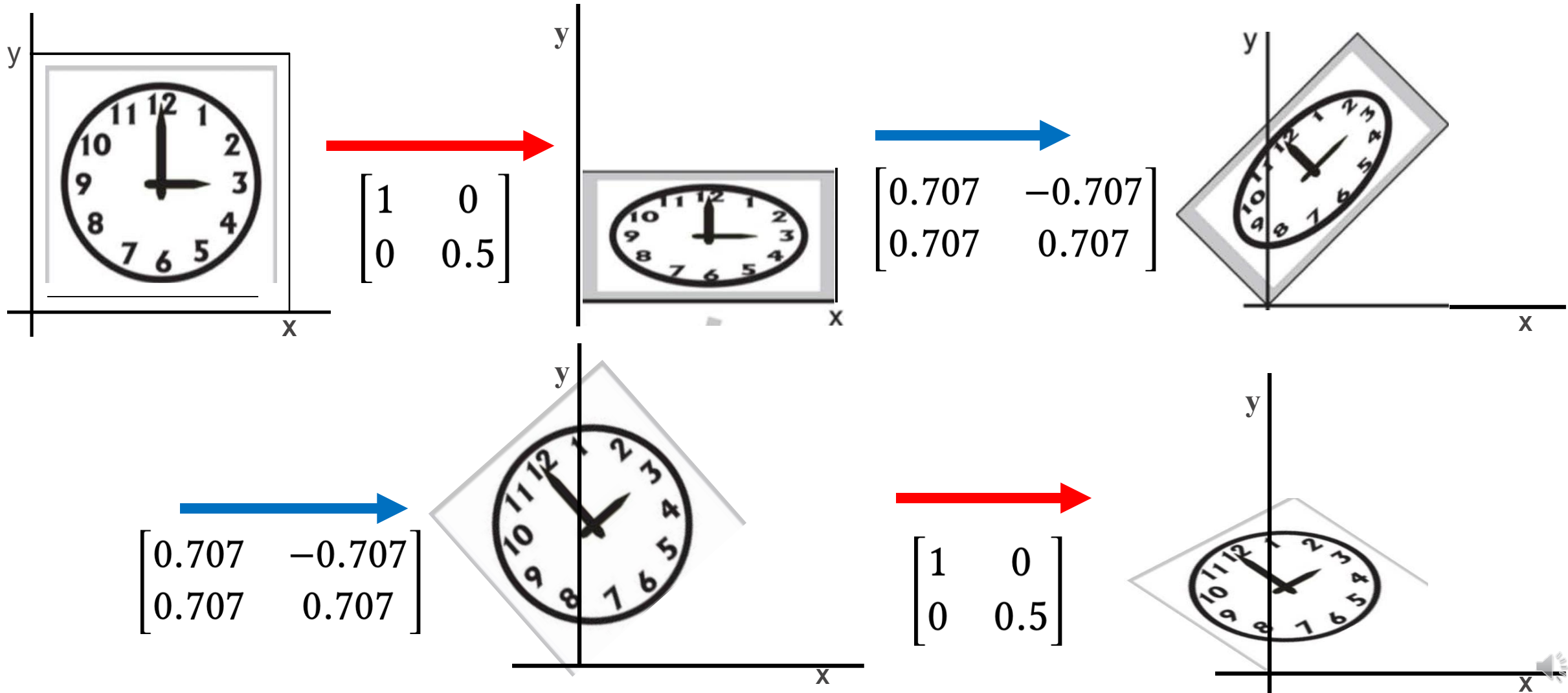


Composing transformations

$$\underbrace{\begin{bmatrix} .707 & -.707 \\ .707 & .707 \end{bmatrix}}_{2^{\text{nd}} \text{ transformation}} \underbrace{\begin{bmatrix} 1.0 & 0 \\ 0 & 0.5 \end{bmatrix}}_{1^{\text{st}} \text{ transformation}} = \begin{bmatrix} .707 & -.353 \\ .707 & .353 \end{bmatrix}$$

Composing Transformations

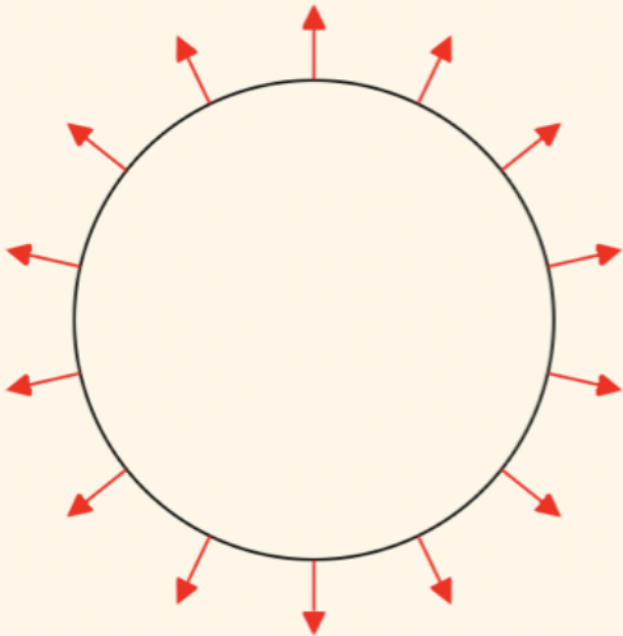
Any sequence of linear transforms can be concatenated into a single 3x3 matrix.
In general transforms DO NOT commute (some special combinations are).



Speaking of vectors, do normals get transformed the same way an object does?

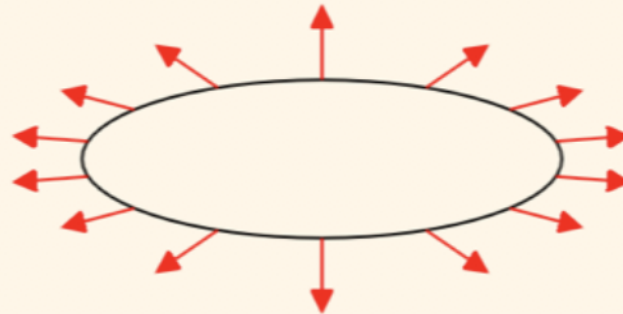
No, thank you for asking.

Original object



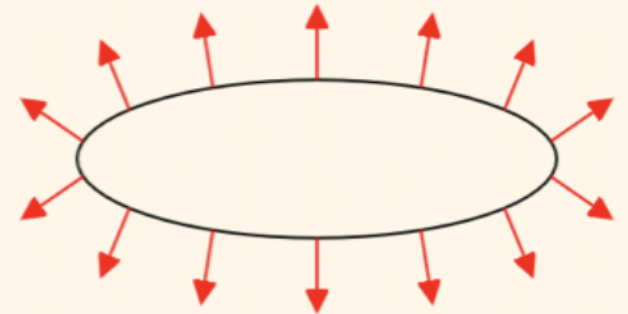
Transformed

With normals using the same transformation matrix



Transformed

With correct transformation applied to normals



What's the right way to transform a normal vector?

$$n'^T t' = 0$$

$$(Xn)^T (Mt) = 0$$

Tangents can be written as the difference of points on an object.
Say $\mathbf{t} = \mathbf{p} - \mathbf{q}$. The transformed tangent $\mathbf{t}' = \mathbf{Mp} - \mathbf{Mq} = \mathbf{M}(\mathbf{p} - \mathbf{q}) = \mathbf{Mt}$.

What's the right way to transform a normal vector?

We want $n^T X^T M t = 0$

So, if $X = (M^{-1})^T$ then,

$$n^T X^T M t = n^T M^{-1} M t = n^T t = 0$$

Representing 3D transforms as a 4x4 matrix

Translate a point $[x \ y \ z]^T$ by $[t_x \ t_y \ t_z]^T$:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Rotate a point $[x \ y \ z]^T$ by an angle t **around z axis**:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos t & -\sin t & 0 & 0 \\ \sin t & \cos t & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Scale a point $[x \ y \ z]^T$ by a factor $[s_x \ s_y \ s_z]^T$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$



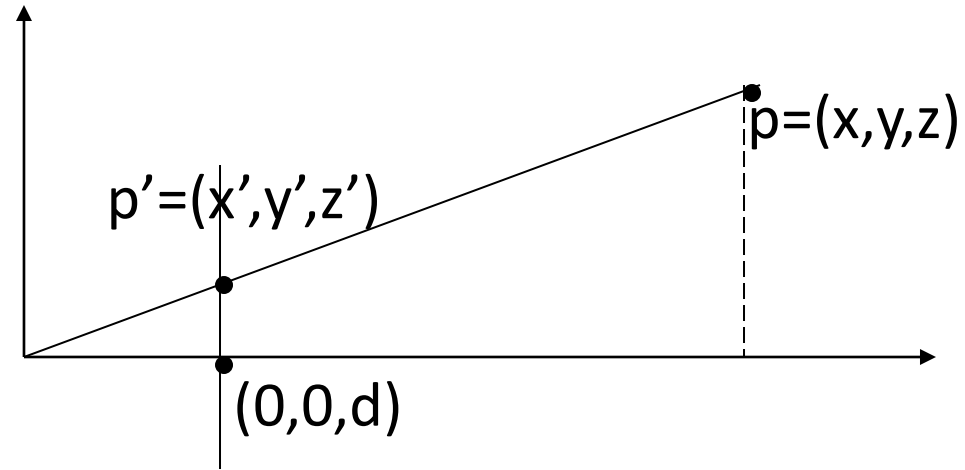


Perspective Projection

$$y' = d * y / z$$

$$x' = d * x / z$$

$$z' = d$$

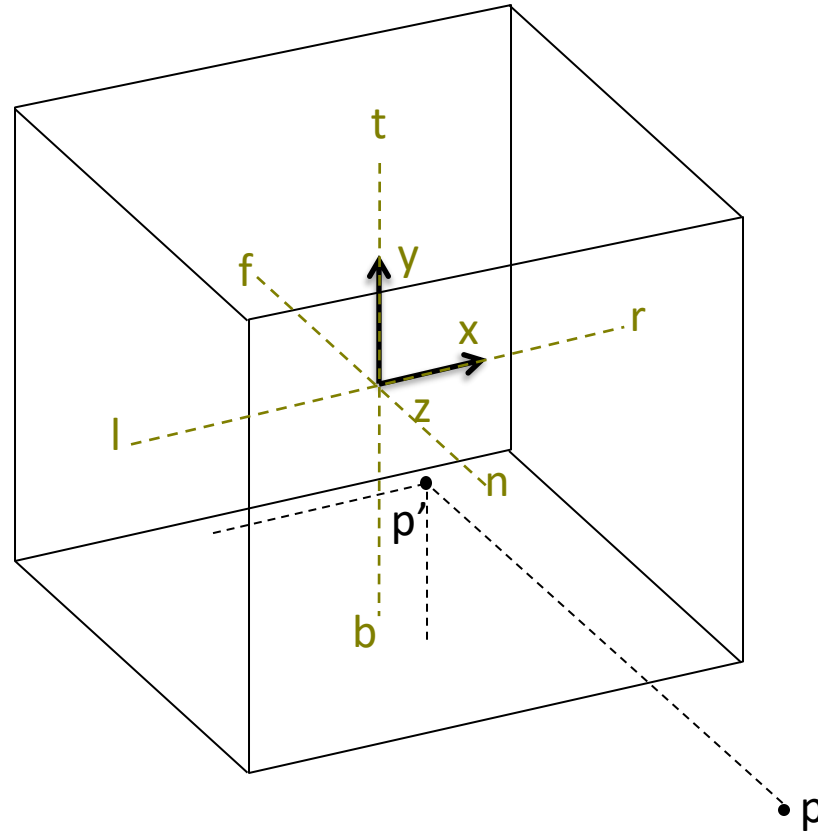


Insight: $w' = z/d$

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$



Canonical view volume

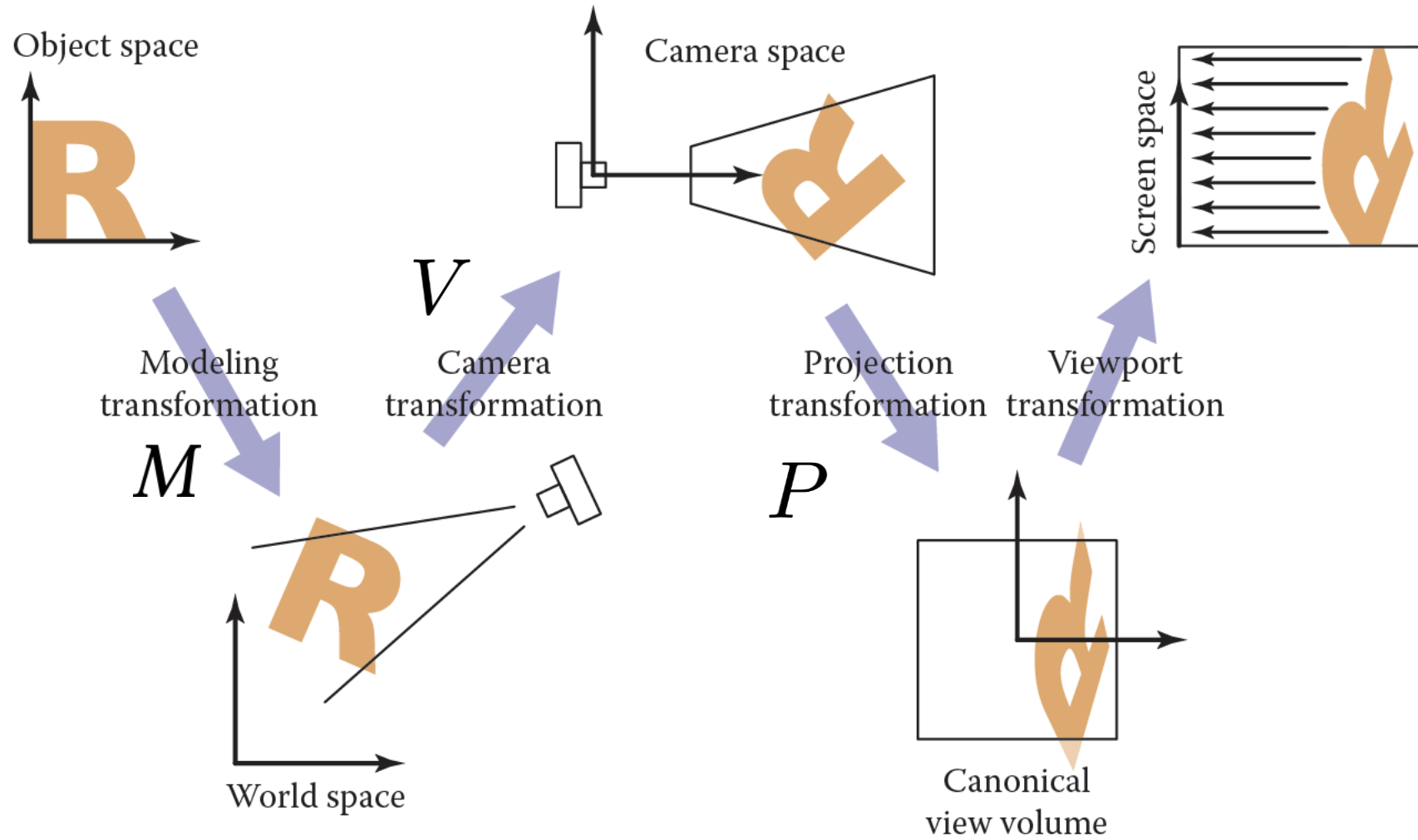


Map a 3D viewing volume
to an origin centered cube
of side length 2.

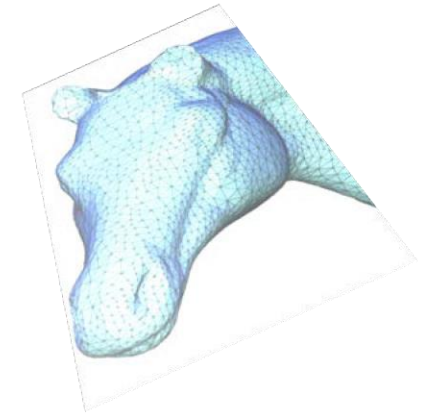
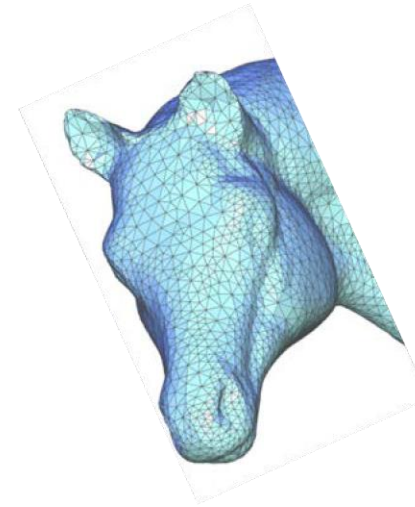
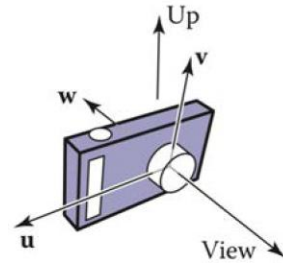
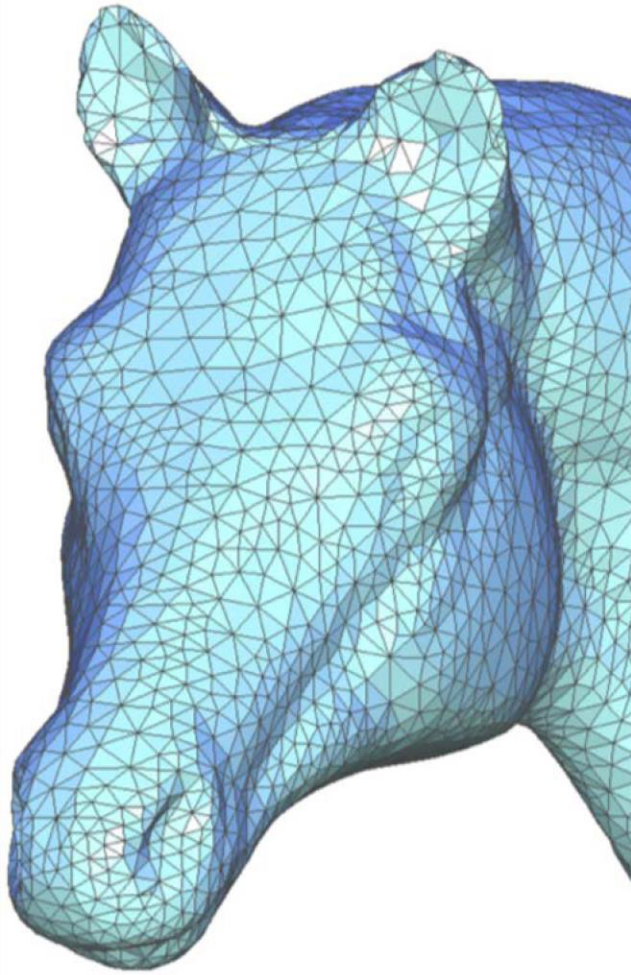
Translate $(-(l+r)/2, -(t+b)/2, -(n+f)/2)$ *
Scale $(2/(r-l), 2/(t-b), 2/(f-n))$



Getting Things Onto The Screen



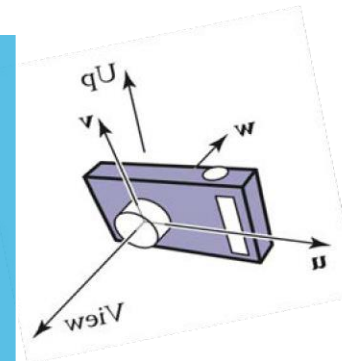
Getting Things Onto The Screen



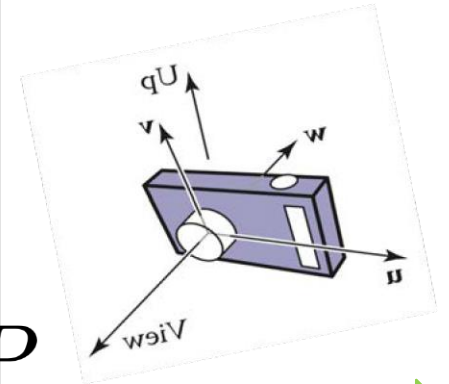
OpenGL combines these into the ModelView Matrix

M

V



P



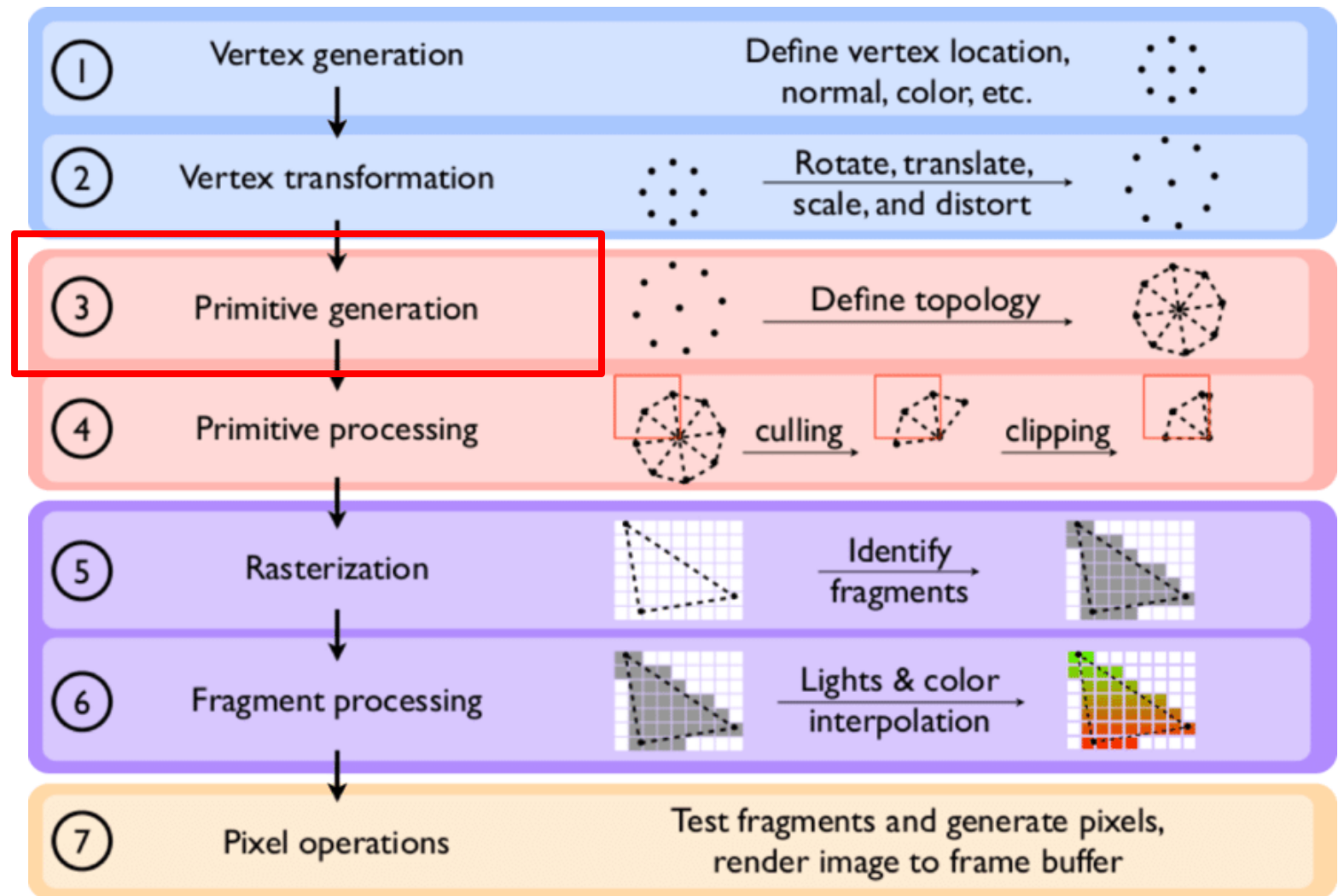
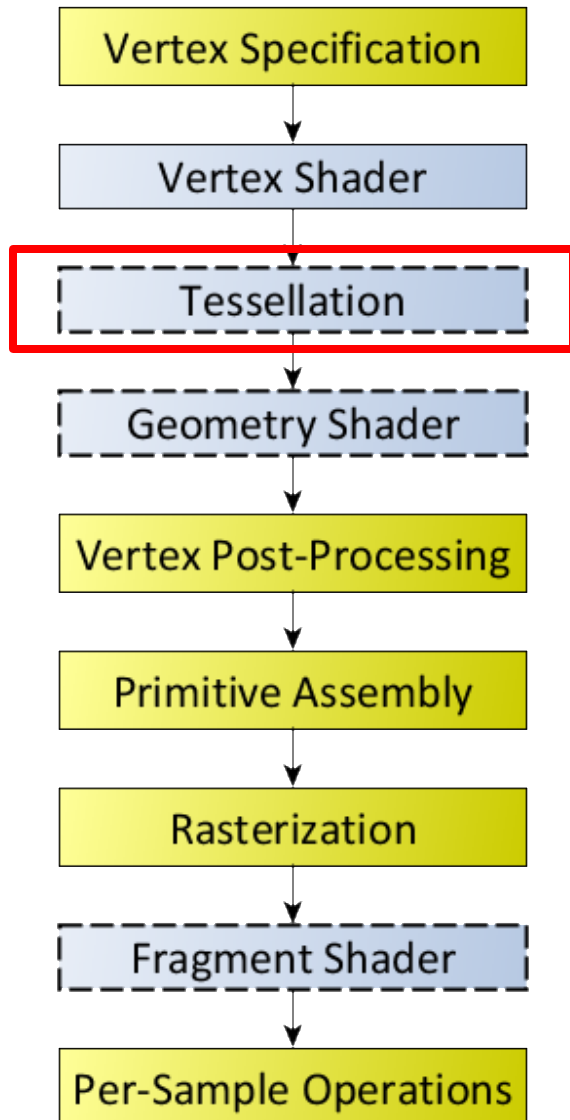
Object Space

World Space

Camera Space

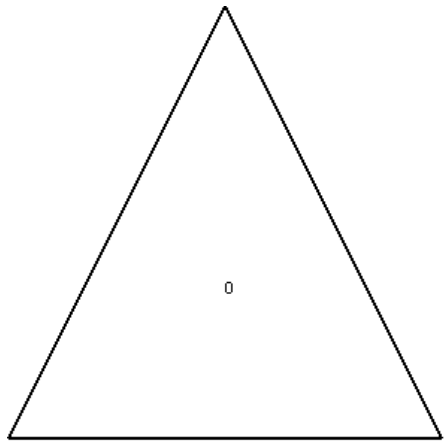
Canonical Space

Modern Graphics Pipeline

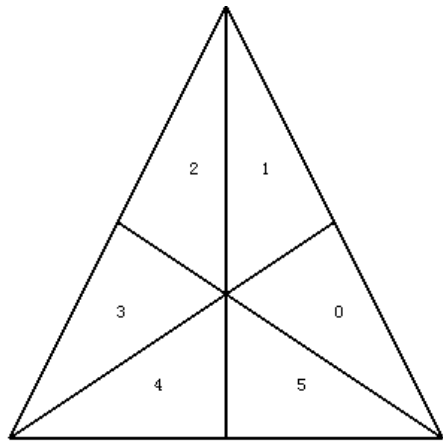


Tessellation Control Shader

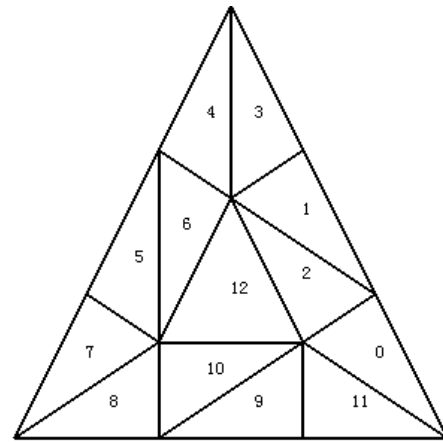
“How much to subdivide a surface”



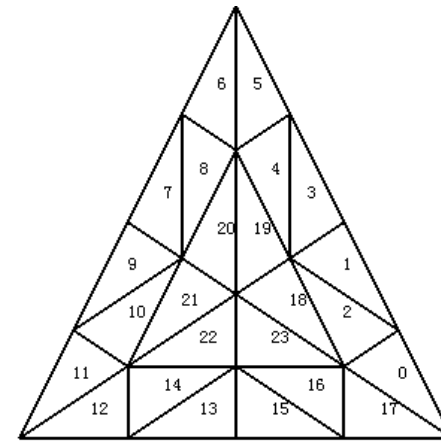
Level 1



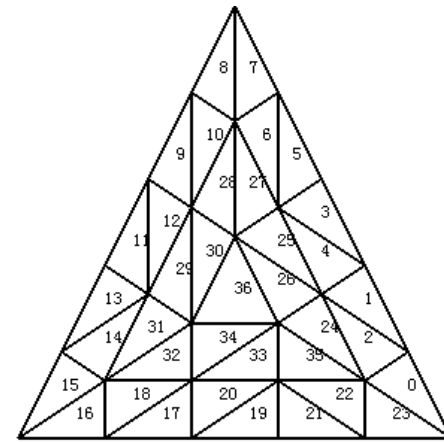
Level 2



Level 3

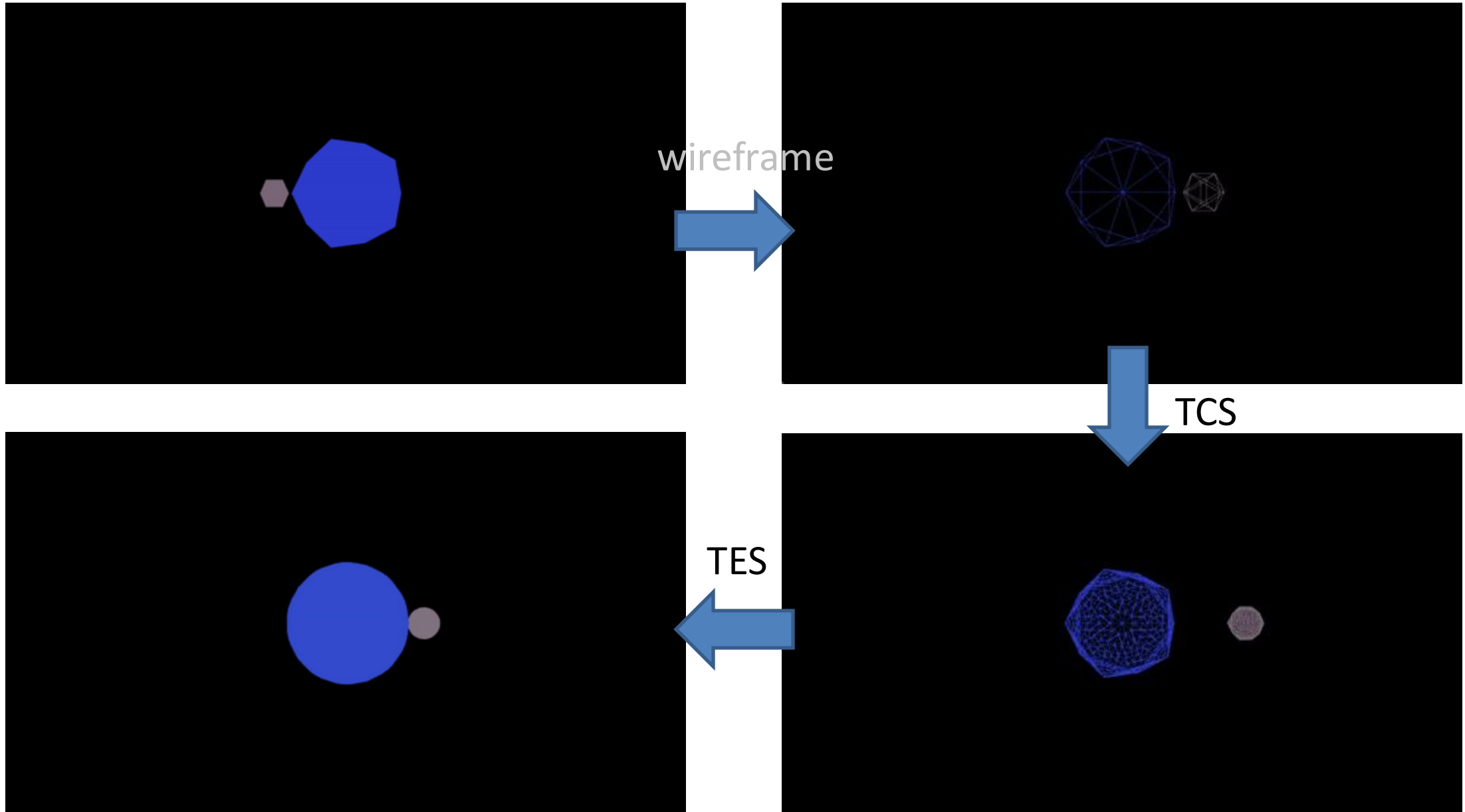


Level 4

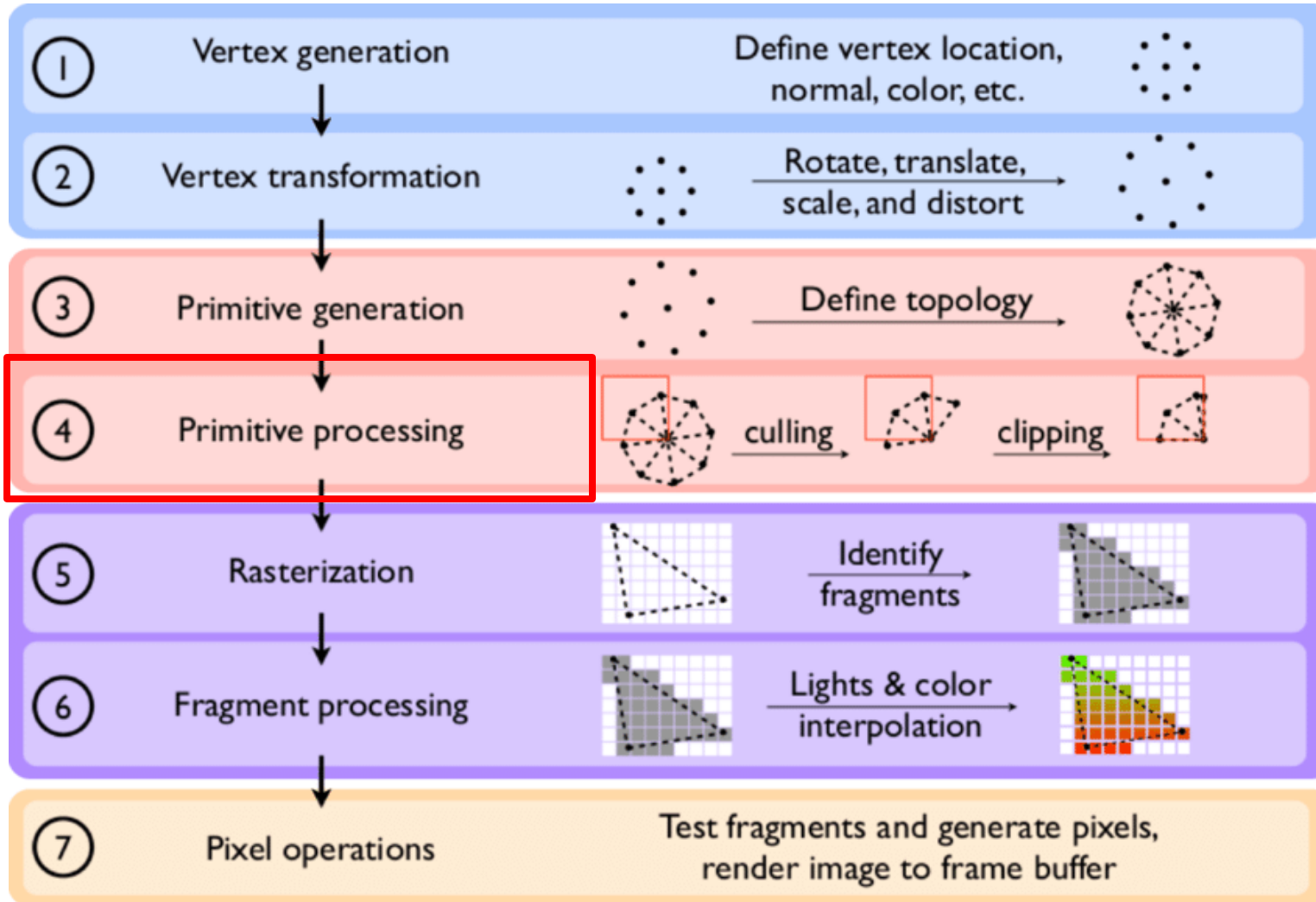


Level 5

Tessellation Evaluation Shader



Modern Graphics Pipeline



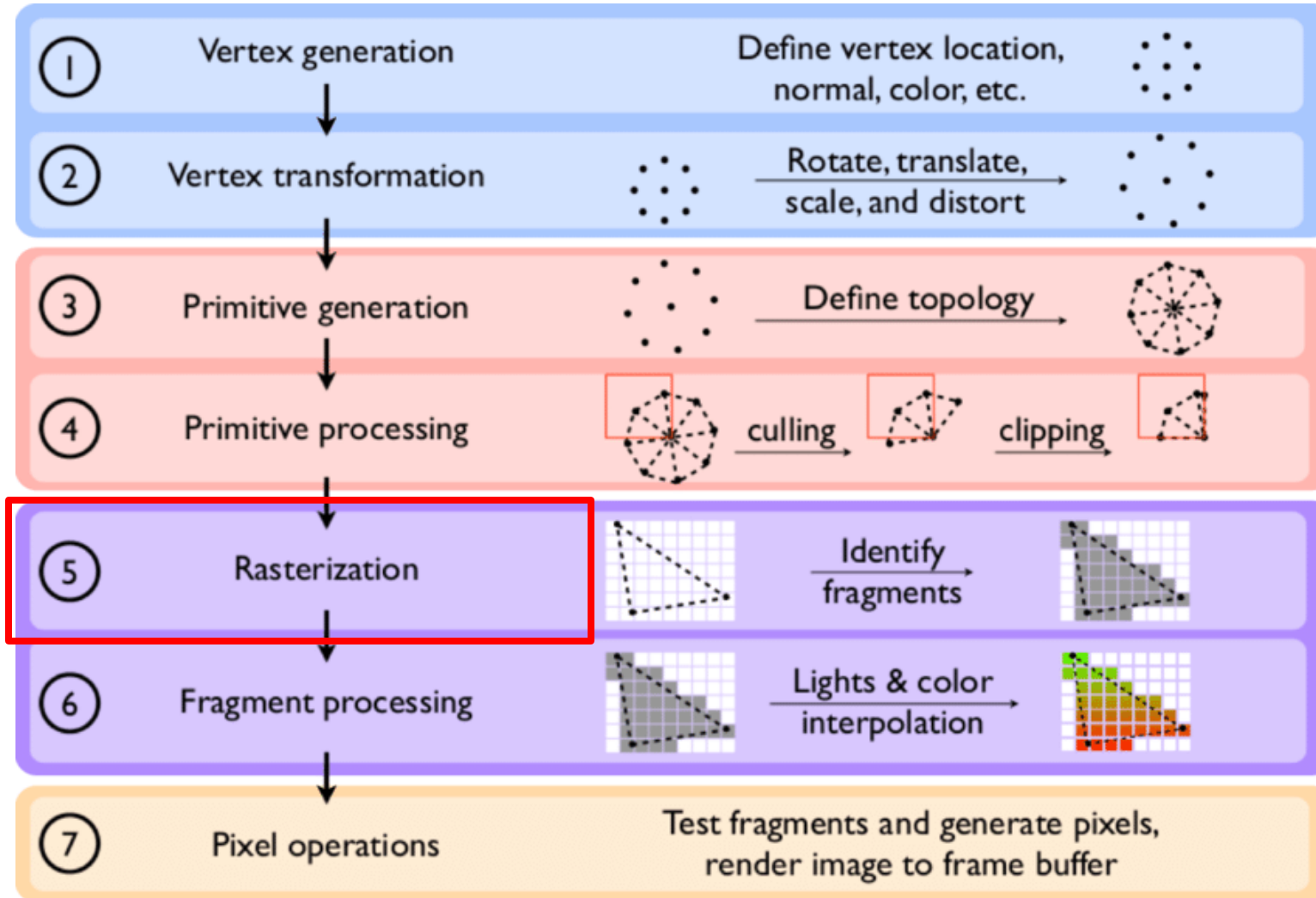
Clipping
Perspective divide and viewport transform to window space

After the perspective projection matrix multiplication in the vertex shader, we need to divide all x, y, z values by w to get them into normalized device coordinates. Then puts things into window space correctly.

<https://www.learnopengles.com/tag/perspective-divide/>

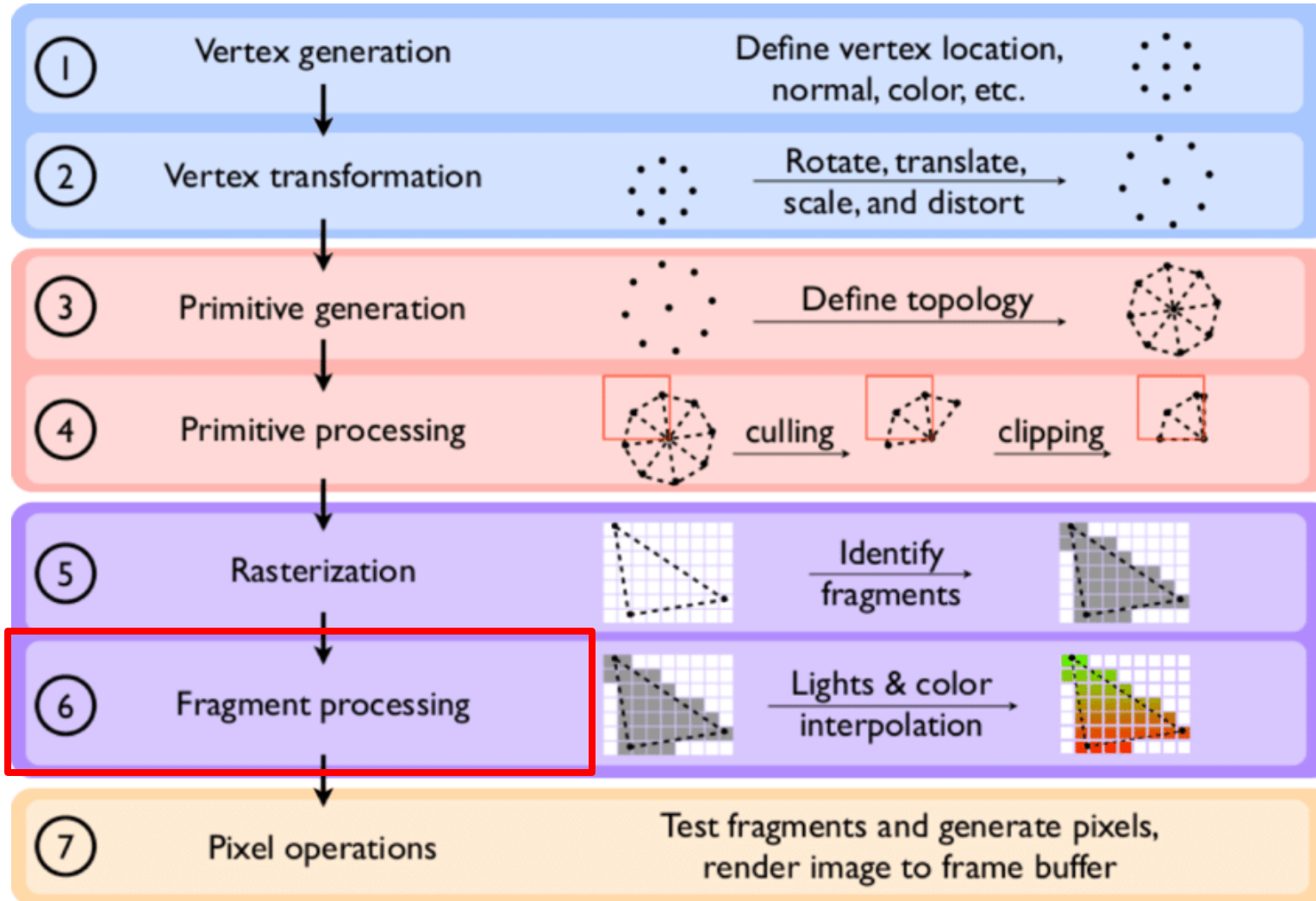
https://www.khronos.org/opengl/wiki/Rendering_Pipeline_Overview

Modern Graphics Pipeline



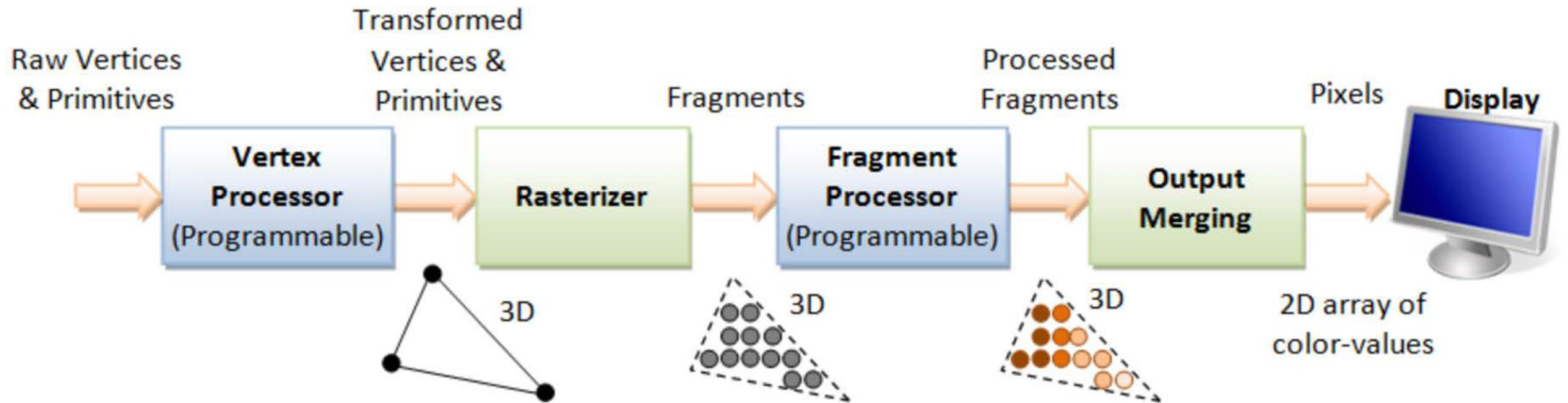
Fragment generation from primitives
At least one fragment for every pixel area covered by primitive being rasterized

Modern Graphics Pipeline

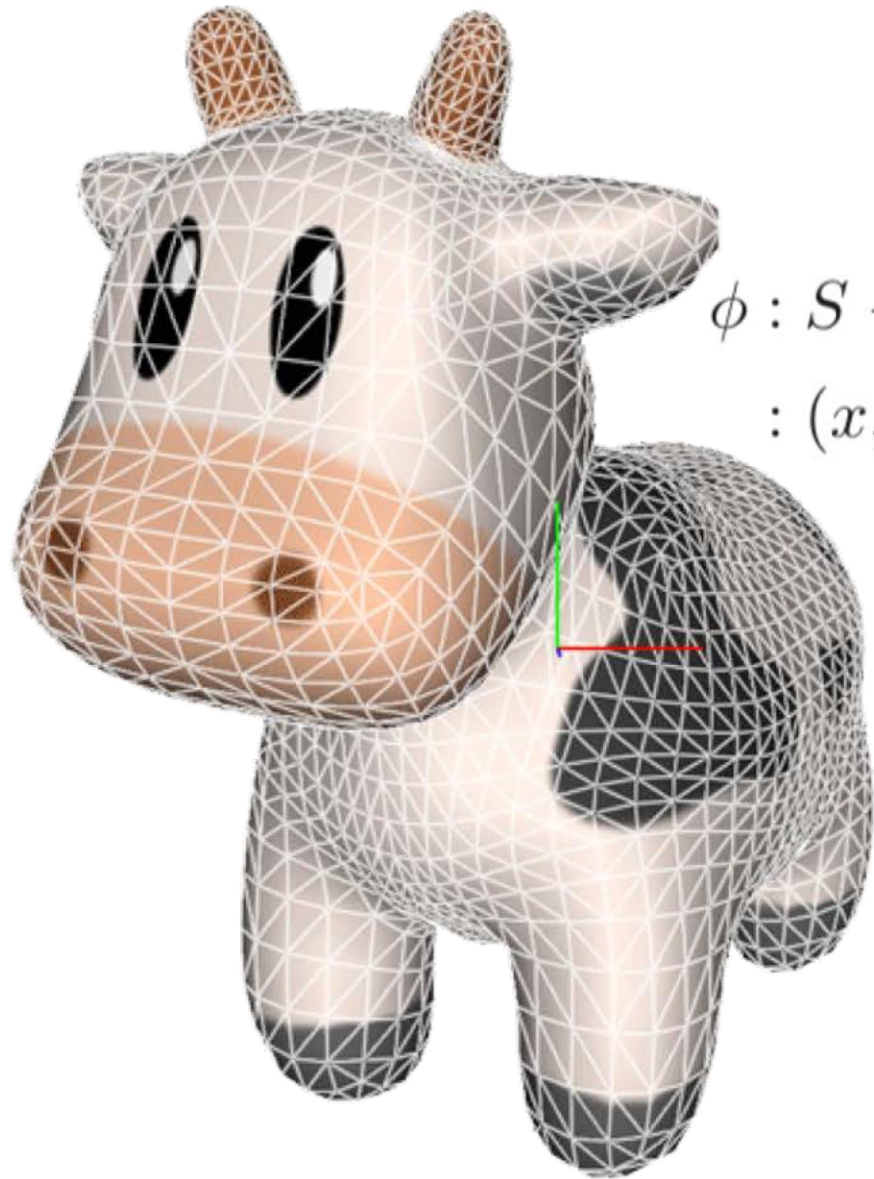


Programmable shader operates on every fragment
Each output fragment has window space position, depth value, and zero or more color values

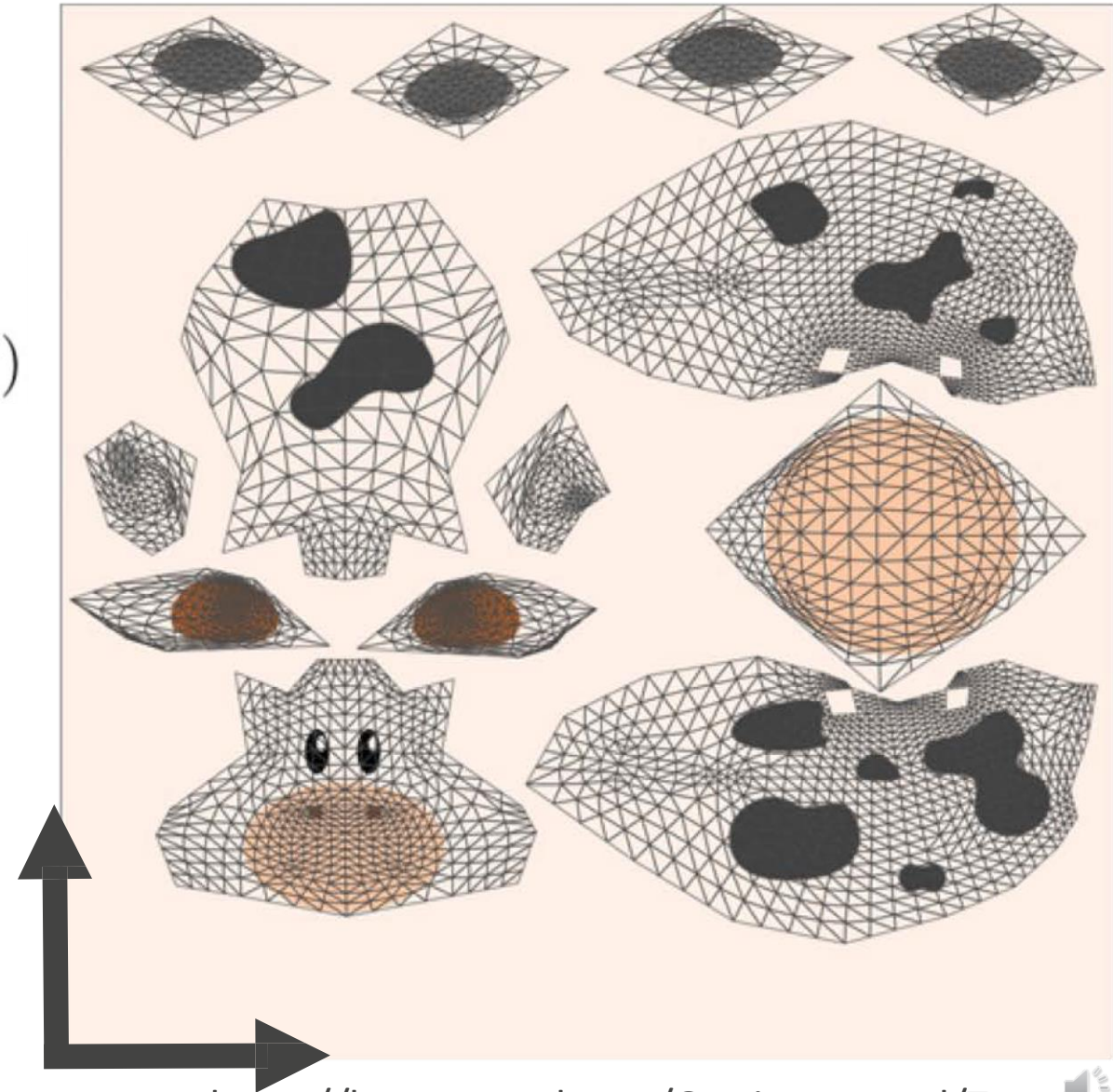




Texture coordinates



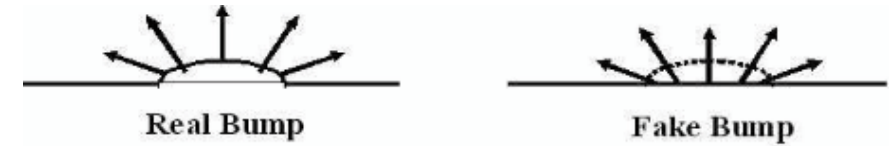
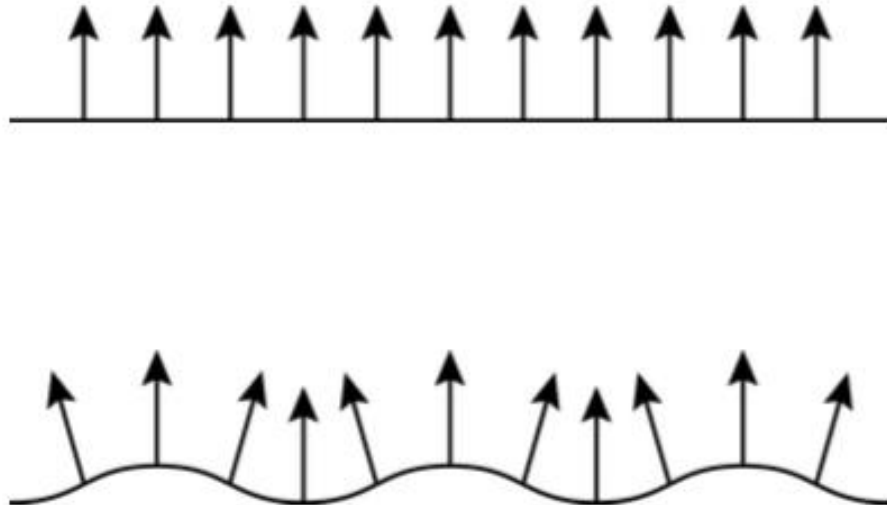
$$\begin{aligned}\phi : S &\rightarrow T \\ &: (x, y, z) \mapsto (u, v)\end{aligned}$$



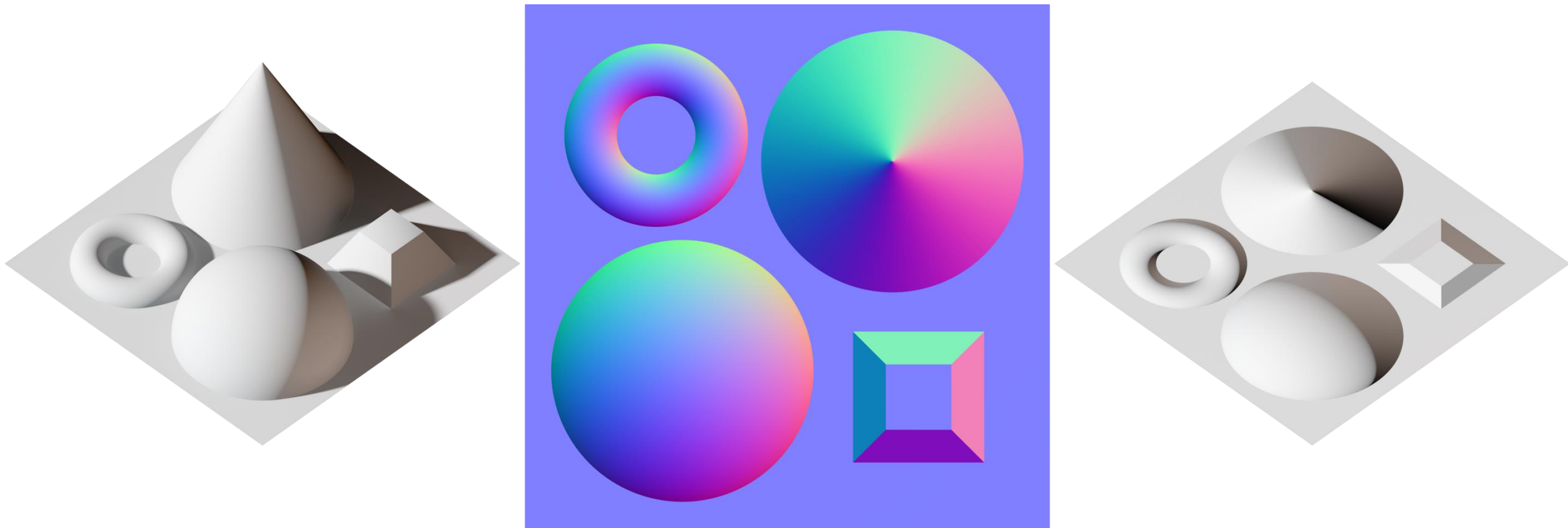
Bump | Normal Mapping

One of the reasons why we apply texture mapping:

Real surfaces are hardly flat but often rough and bumpy. These bumps cause (slightly) different reflections of the light.



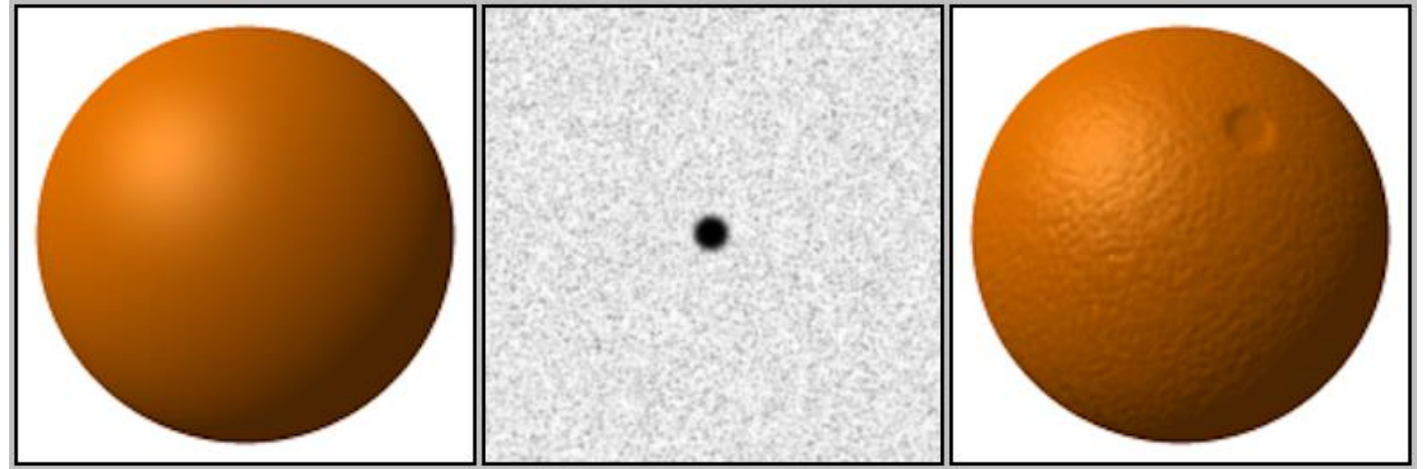
Normal Mapping



“here’s exactly which way the surface is tilted.”



Bump Mapping



Bump map $B(u,v)$

Compute the perceived surface normal for the surface displaced by the bump value $B(u,v)$ along its normal $n(u,v)$.

Displaced surface point $p_d(u,v) = p(u,v) + B(u,v) * n(u,v)$

Displaced surface normal $n_d(u,v) = p'^u_d(u,v) \times p'^v_d(u,v)$

“this part is higher/lower than its neighbor” → compute slope.



Bump Mapping

Compute the perceived surface normal for the surface displaced by the bump value $B(u,v)$ along its normal $n(u,v)$.

Displaced surface point $p_d(u,v) = p(u,v) + B(u,v)*n(u,v)$

Displaced surface normal $n_d(u,v) = p'^u_d(u,v) \times p'^v_d(u,v)$

$B(u,v)$ is small

$$p'^u_d(u,v) = p'^u(u,v) + B'^u(u,v)*n(u,v) + \cancel{B(u,v)*n'^u(u,v)}$$

$$n_d = (p'^u + B'^u n) \times (p'^v + B'^v n) \quad // \text{ not showing } (u,v)$$

$$n_d = p'^u \times p'^v + p'^u \times n B'^v + B'^u n \times p'^v + B'^u B'^v n \times n$$

$$n_d = n + B'^v (p'^u \times n) + B'^u (n \times p'^v)$$

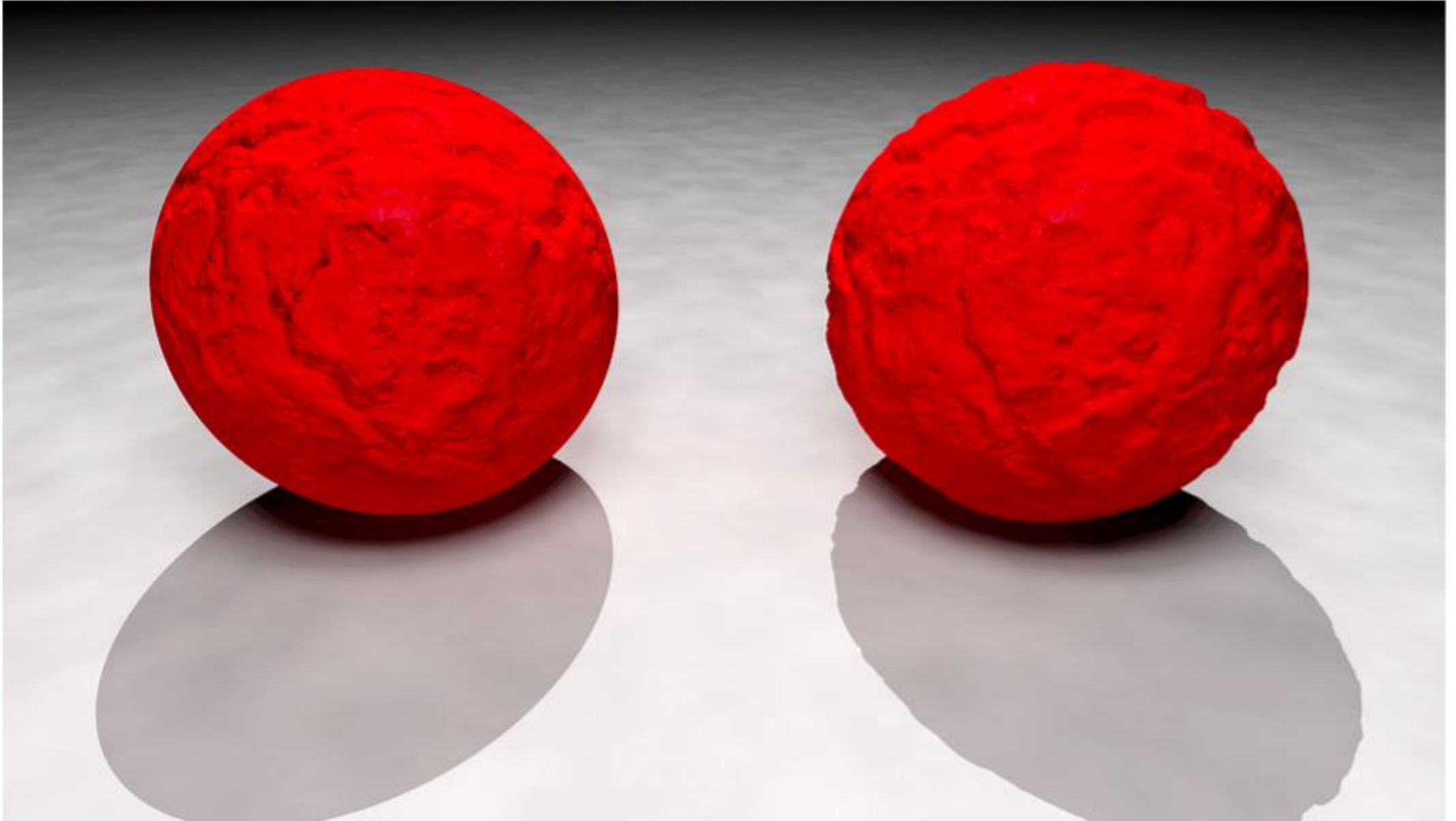
B' can be precomputed by finite difference and stored as a texture.

p' can be computed analytically or by finite difference.

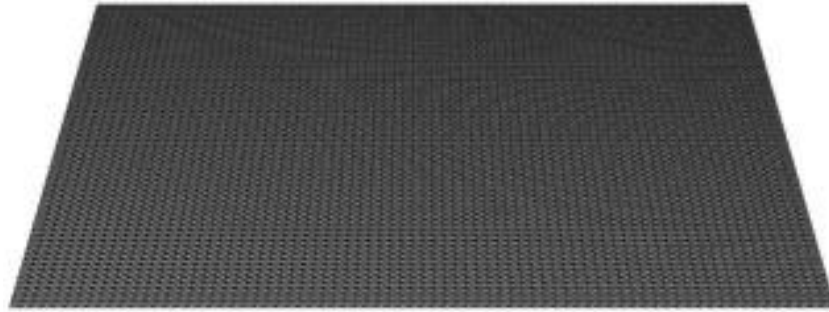


Bump | Normal Mapping vs. Geometry

Major problems with bump mapping: silhouettes and shadows



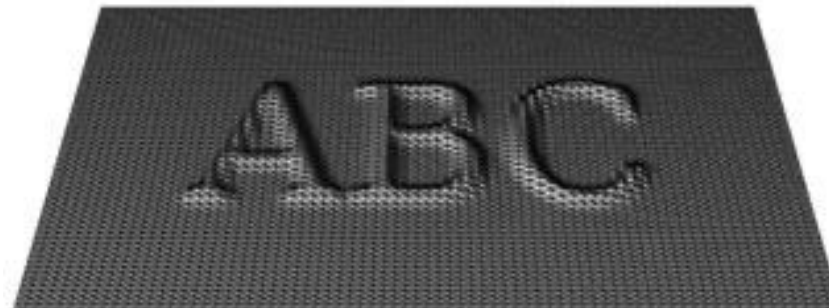
Displacement (Bump) Mapping



ORIGINAL MESH



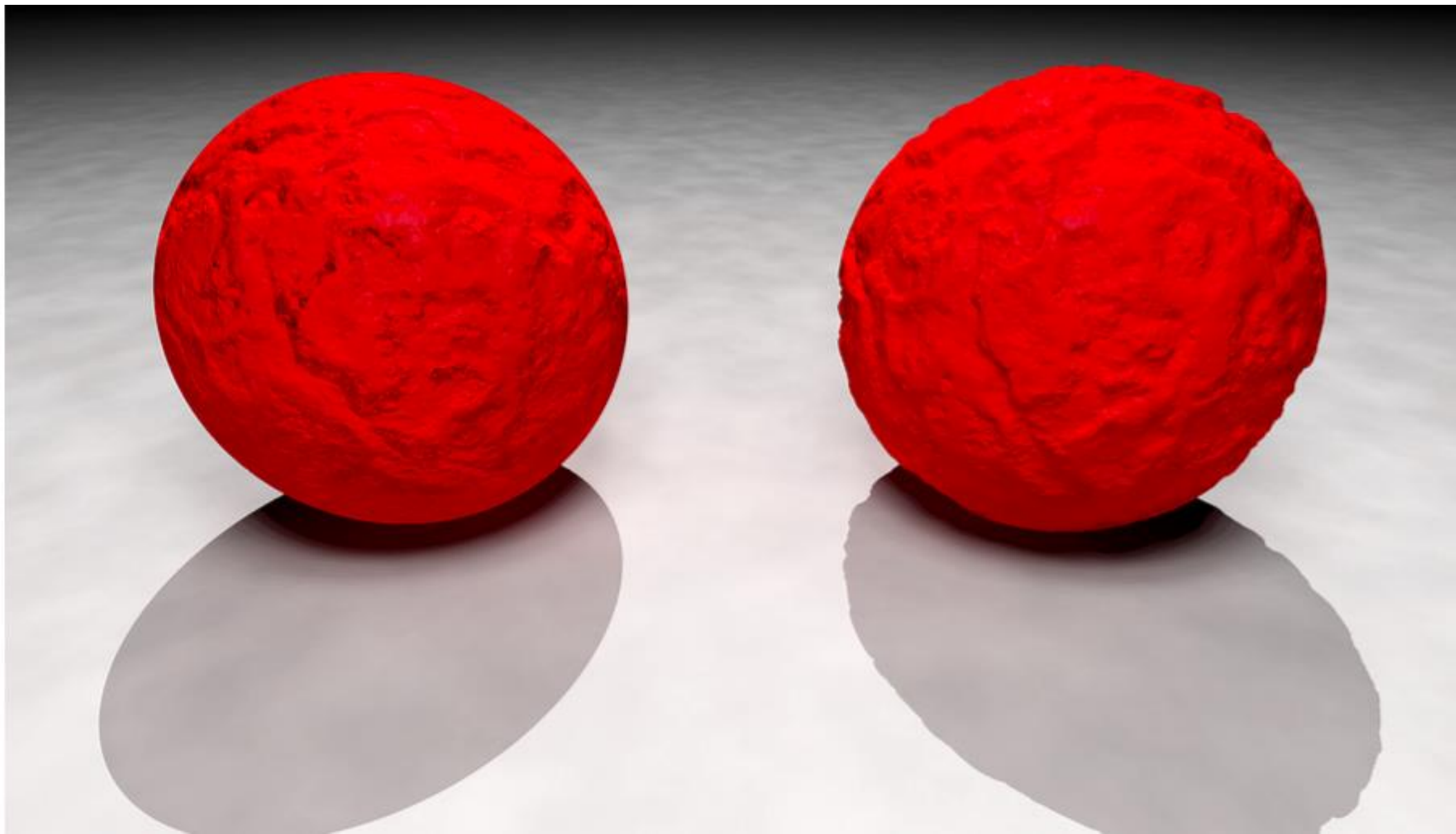
DISPLACEMENT MAP



MESH WITH DISPLACEMENT



Bump | Normal Mapping vs. Displacement Mapping

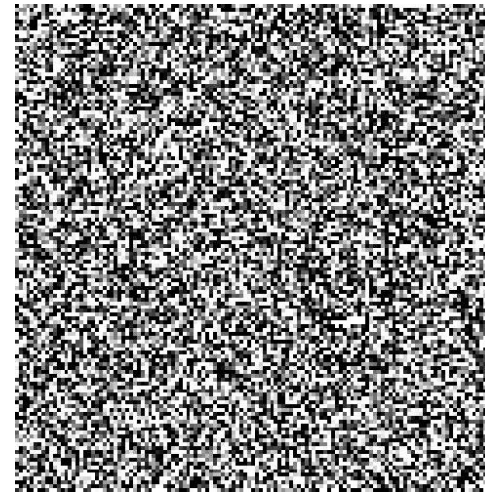


Perlin Noise



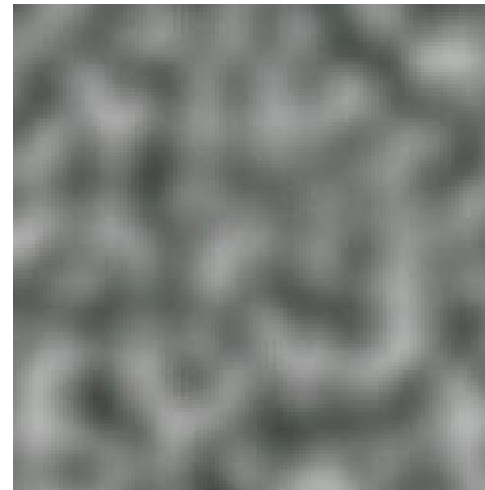
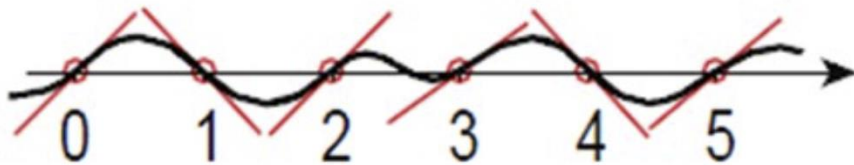
Properties of noise:

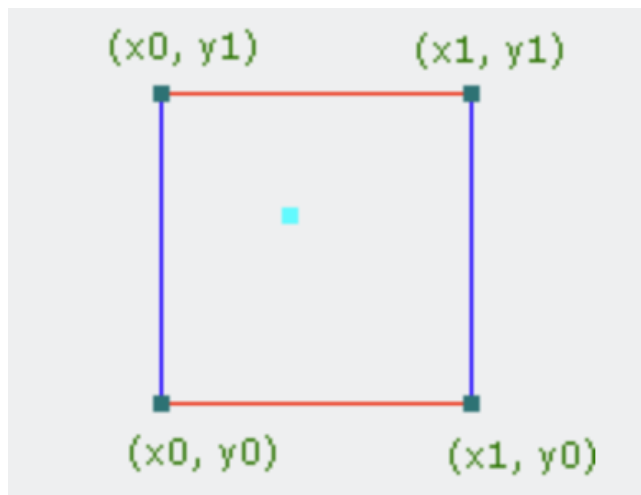
- well-defined everywhere in space.
- It randomly varies between -1 and 1.
- Its frequency is band-limited.



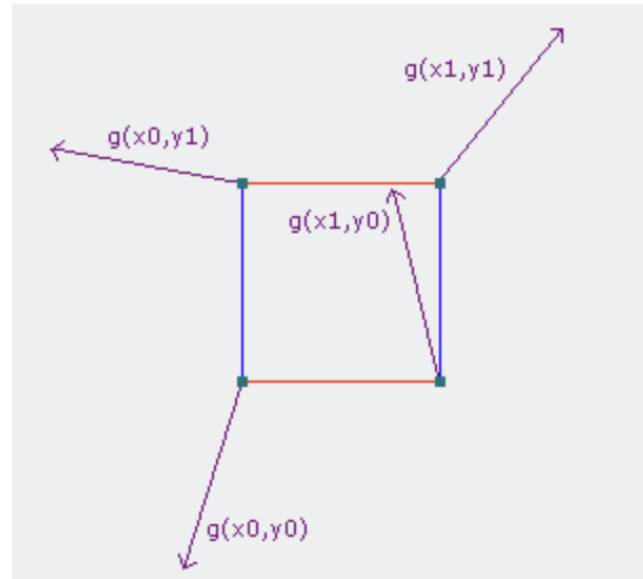
Gradient Noise in 1D:

- Define a pseudo-random gradient direction $\mathbf{d}_i$ at each integer point i .
- The value at a point $\mathbf{x}$ relative to an integer neighbour i is $(\mathbf{x}-i) \cdot \mathbf{d}_i$.
- Smoothly interpolate between neighboring values for $\mathbf{n}(\mathbf{x})$.





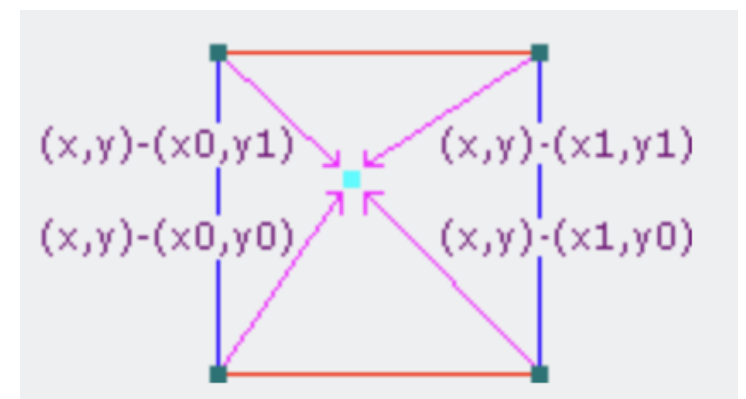
1) Create a grid



2) Create pseudo-random grad vectors per vertex

4) dot(**grad**, **disp**) for each pair of vectors

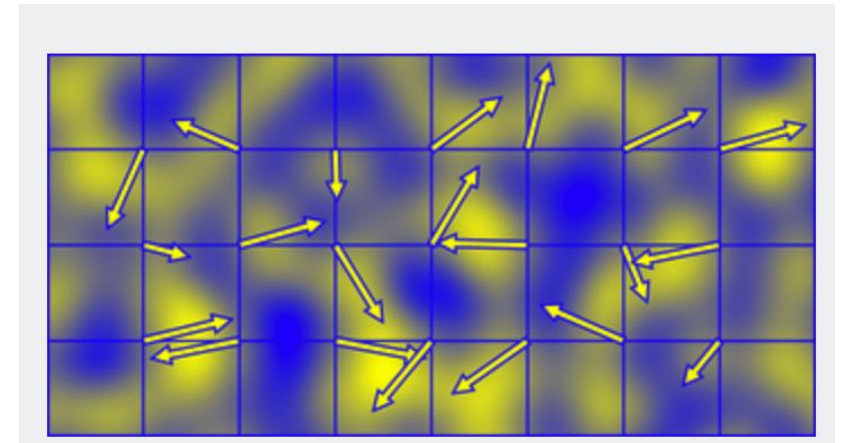
5) Average the 4 dot products

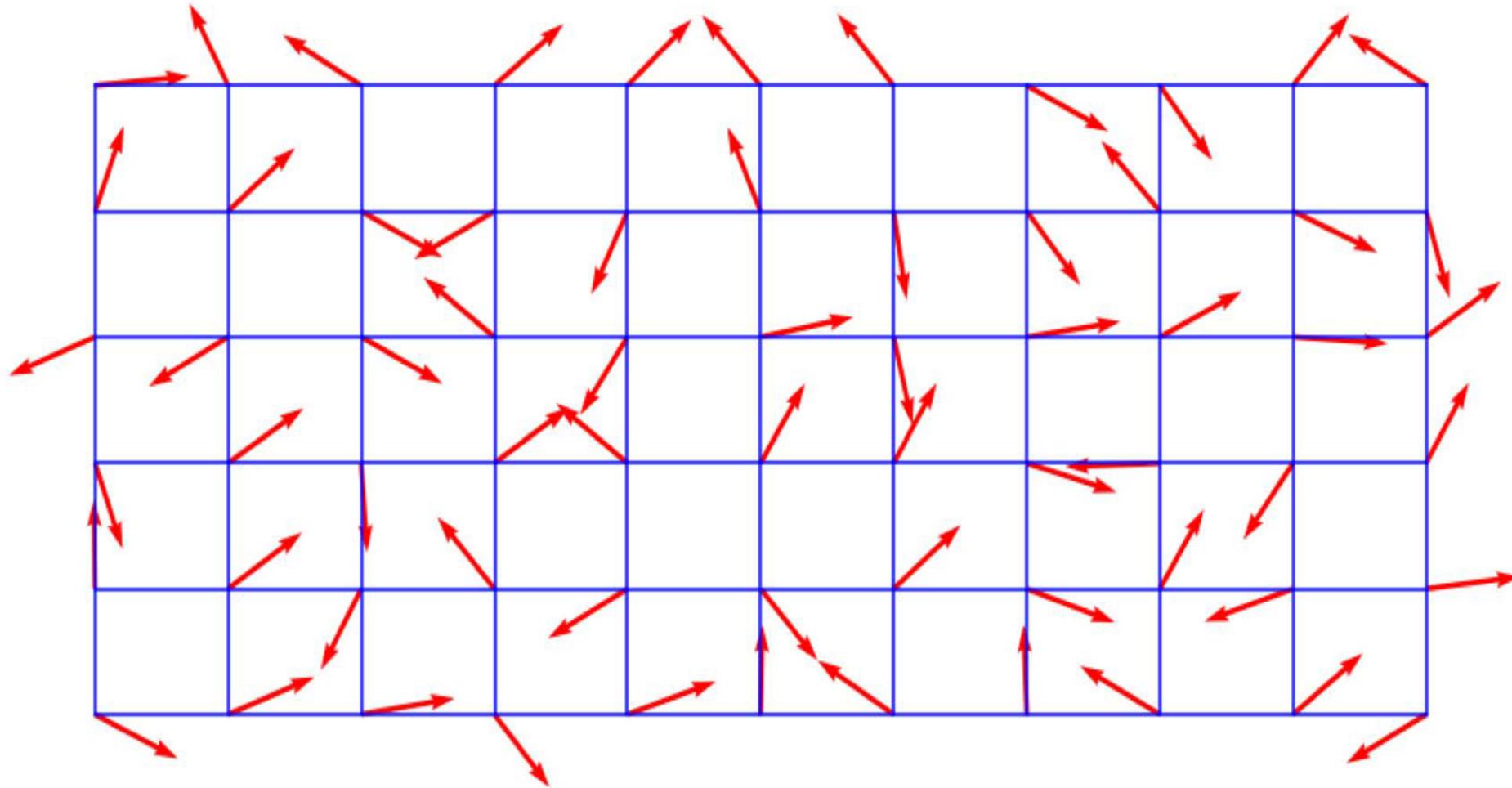


3) For a sample point, find displacement vector

6) Pass value into an ease curve.

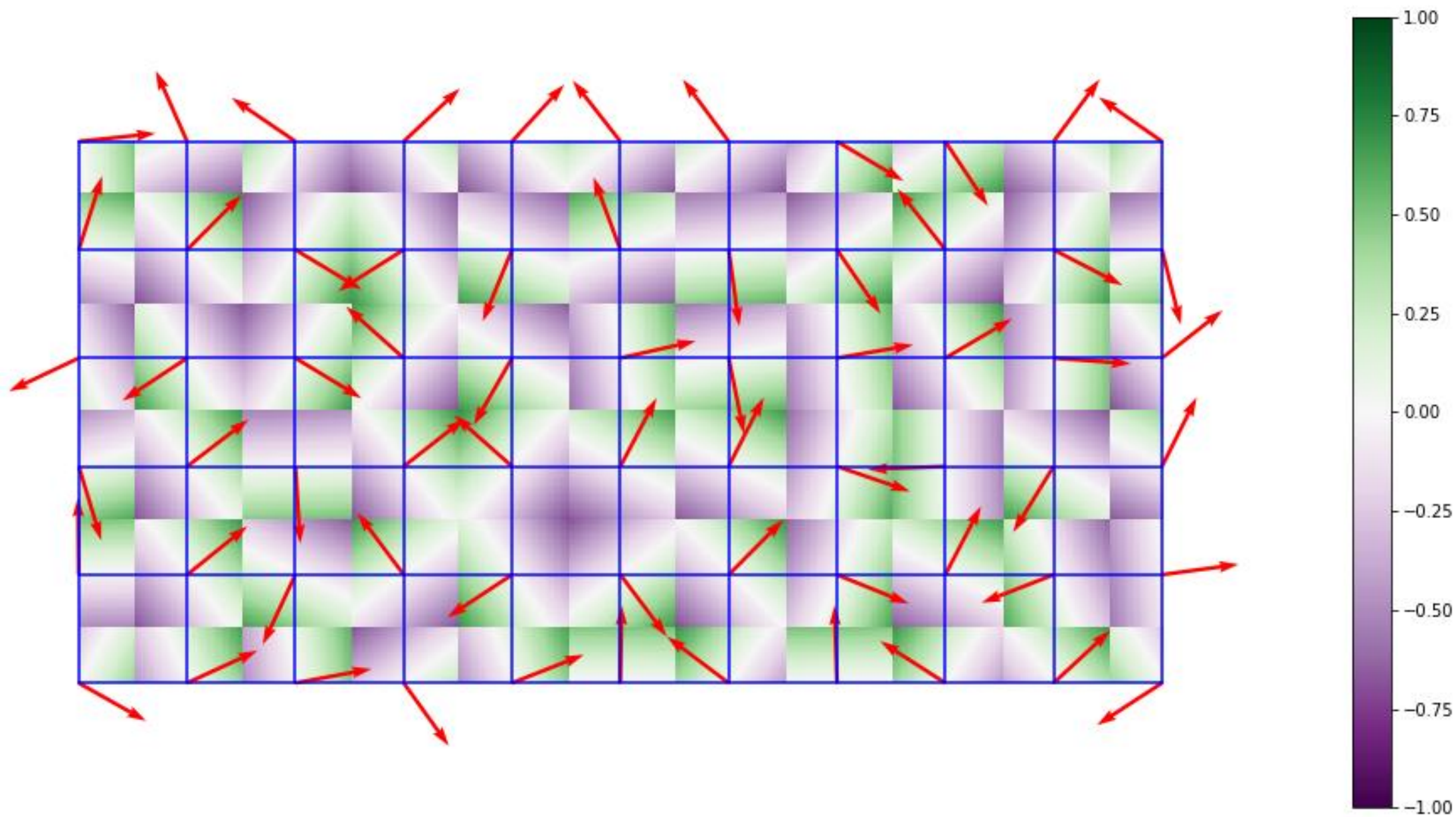
$$6t^5 - 15t^4 + 10t^3$$

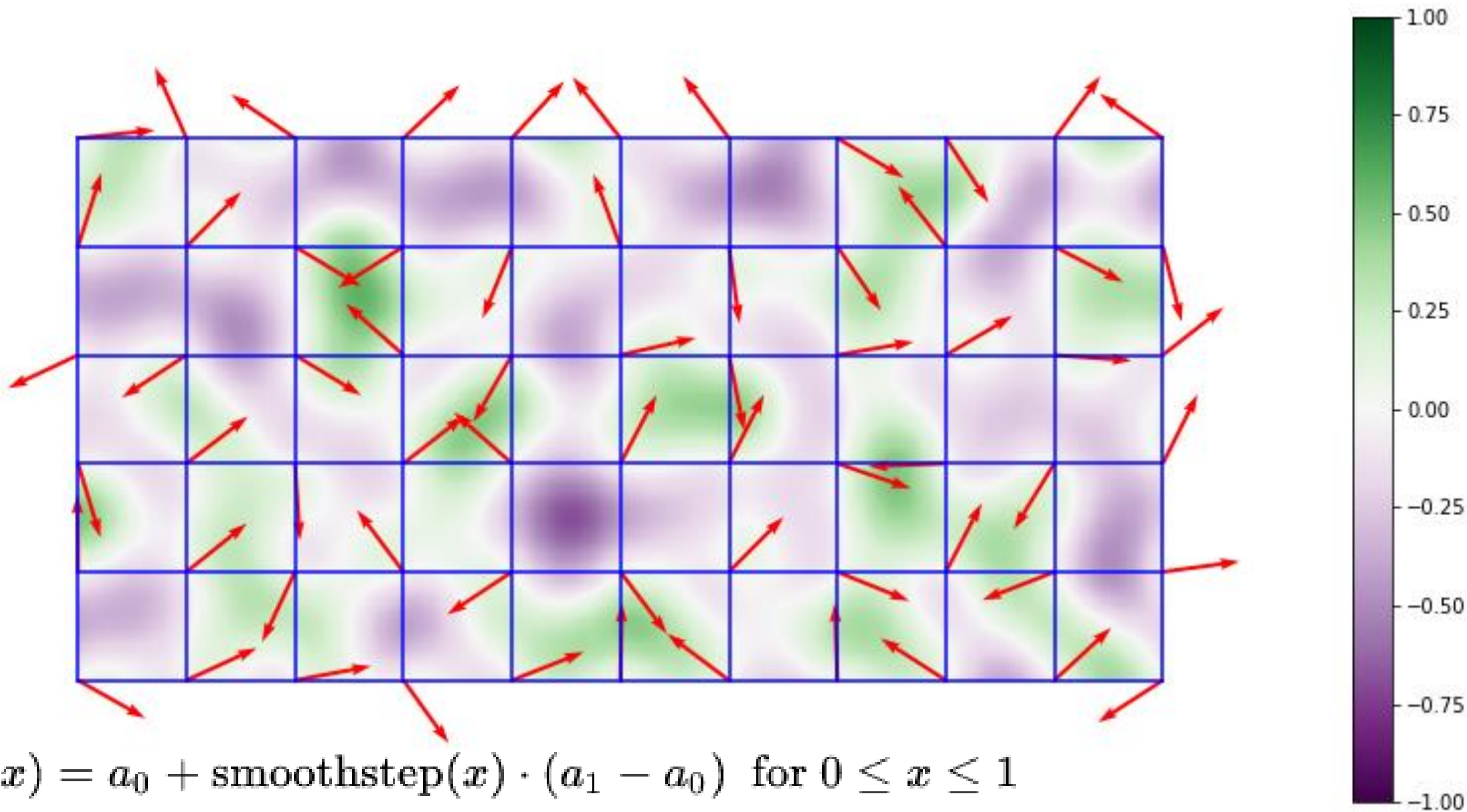




https://en.wikipedia.org/wiki/Perlin_noise

<https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/perlin-noise-part-2/perlin-noise>





Assignment #6: Tasks

src/identity.glsl build and return an identity matrix.

src/uniform_scale.glsl

src/translate.glsl, src/rotate_about_y.glsl

src/model.glsl, src/model_view_projection.vs

src/blue_and_gray.fs

With these implemented you should now be able to run `./shaderpipeline ../data/test-02.json` and see an animation of a gray moon orbiting around a blue planet. If you press L this should switch to a wireframe rendering.

src/5.tcs, snap_to_sphere.tes

Running `./shaderpipeline ../data/test-03.json` and pressing L should produce an animation of a gray moon orbiting around a blue planet in wireframe with more triangles:

blinn_phong.glsl, lit.fs

Running `./shaderpipeline ../data/test-05.json` adds light to the scene and we see a smooth appearance with specular highlights:

random_direction.glsl, smooth_step.glsl, perlin_noise.glsl, procedural_color.glsl

Be creative! Your procedural colored shape does not need to look like marble specifically and does not need to match the example. Mix and match different noise frequencies and use function composition to create an interesting, complex pattern.

Be creative! Your planets do not need to look like the earth/moon and do not need to look like the example planets.

Hint: Sprinkle noise on everything: diffuse color, specular color, normals, specular exponents, color over time.

```
mat4 uniform_scale(float s)
{ return mat4(  .., .., .., ..,
               .., .., .., ..,
               .., .., .., ..,
               .., .., .., ..); }
```


Next: Animation

